

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



Nguyễn Quang Ninh

**NGHIÊN CỨU, ĐÁNH GIÁ HIỆU NĂNG SLLMS
TRONG BÀI TOÁN SINH CÂU HỎI TRẮC NGHIỆM
TỪ ẢNH VÀ XÂY DỰNG ỨNG DỤNG THỰC NGHIỆM**

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Công nghệ thông tin

HÀ NỘI - 2026

**ĐẠI HỌC QUỐC GIA HÀ NỘI
TRƯỜNG ĐẠI HỌC CÔNG NGHỆ**



NGUYỄN QUANG NINH

**NGHIÊN CỨU, ĐÁNH GIÁ HIỆU NĂNG SLLMS
TRONG BÀI TOÁN SINH CÂU HỎI TRẮC NGHIỆM
TỪ ẢNH VÀ XÂY DỰNG ỨNG DỤNG THỰC NGHIỆM**

KHOÁ LUẬN TỐT NGHIỆP

Ngành: Công nghệ thông tin

Cán bộ hướng dẫn: TS. Nguyễn Văn Sơn

HÀ NỘI - 2026

**VIETNAM NATIONAL UNIVERSITY, HANOI
UNIVERSITY OF ENGINEERING AND TECHNOLOGY**



Nguyen Quang Ninh

**RESEARCH AND EVALUATION OF SLLMS PERFORMANCE IN
IMAGE-BASED MULTIPLE-CHOICE QUESTION GENERATION
AND EXPERIMENTAL APPLICATION DEVELOPMENT**

BACHELOR'S THESIS

Major: Information Technology

Supervisor: Dr. Nguyen Van Sơn

HANOI - 2026

Lời cảm ơn

Lời đầu tiên, tôi xin gửi lời cảm ơn chân thành và lòng biết ơn sâu sắc tới thầy Nguyễn Văn Sơn, người đã tận tình hướng dẫn tôi trong suốt thời gian thực hiện nghiên cứu này.

Tôi cũng xin gửi lời cảm ơn chân thành tới các thầy, cô hiện đang công tác và giảng dạy tại trường Đại học Công nghệ - Đại học Quốc gia Hà Nội đã tạo mọi điều kiện thuận lợi cho chúng tôi học tập và nghiên cứu.

Lời cam đoan

Tôi là Nguyễn Quang Ninh, sinh viên lớp QH-2022-I/CQ-I-IT15 khóa 67 theo học ngành Công nghệ thông tin tại trường Đại học Công Nghệ - Đại học Quốc gia Hà Nội. Tôi xin cam đoan khóa luận “Nghiên cứu, đánh giá hiệu năng sLLMs trong bài toán sinh câu hỏi trắc nghiệm từ ảnh và xây dựng ứng dụng thực nghiệm” là công trình nghiên cứu do bản thân tôi thực hiện. Các nội dung nghiên cứu, kết quả trong báo cáo là xác thực. Các thông tin sử dụng trong báo cáo là có cơ sở và không có nội dung nào sao chép từ các tài liệu mà không ghi rõ trích dẫn tham khảo. Tôi xin chịu trách nhiệm về lời cam đoan này.

Hà Nội, ngày XX tháng XX năm 2026

Sinh viên

Nguyễn Quang Ninh

Tóm tắt

Việc tự động sinh câu hỏi trắc nghiệm từ tài liệu giáo dục là nhu cầu cấp thiết trong bối cảnh số hóa giáo dục, nhưng các giải pháp hiện tại phụ thuộc vào mô hình ngôn ngữ lớn thương mại với chi phí cao và yêu cầu kết nối internet ổn định. Khóa luận này đề xuất kiến trúc đa tác tử, sử dụng các mô hình ngôn ngữ lớn cỡ nhỏ (sLLM - Small Large Language Model) để sinh câu hỏi trắc nghiệm chất lượng cao từ ảnh chụp văn bản. Hệ thống triển khai ba tác tử *sinh câu hỏi* hoạt động song song theo từng cấp độ Bloom, kết hợp với tác tử *Phân loại* để kiểm chứng cấp độ nhận thức, tác tử *Học sinh* để kiểm tra khả năng giải được và tác tử *Hiệu chỉnh* để sửa các phương án trả lời chưa đạt yêu cầu. Khung đánh giá ba tiêu chí gồm khả năng giải được dựa trên ngữ cảnh, chất lượng phương án nhiễu và độ bám sát phân loại nhận thức, kết hợp với phương pháp sử dụng LLM để đánh giá (LLM-as-a-Judge), được xây dựng để đánh giá tự động chất lượng câu hỏi. Ngoài ra, quy trình sinh dữ liệu OCR sử dụng kỹ thuật tạo ảnh tổng hợp bằng trình duyệt nhằm phát triển bộ dữ liệu đánh giá. Kết quả thực nghiệm cho thấy kiến trúc đa tác tử cải thiện chất lượng sLLM mạnh mẽ, với mức tăng chỉ số chất lượng lên tới 93% đối với các mô hình yếu, đồng thời duy trì chi phí dưới \$0.50 cho mỗi 100 đoạn văn. Mô hình sLLM tốt nhất là Gemma 3 kết hợp đa tác tử đạt hiệu suất cạnh tranh với mô hình thương mại ở các cấp độ Bloom. Ứng dụng Openlet được phát triển để minh họa khả năng ứng dụng thực tế của hệ thống.

Từ khóa: Mô hình ngôn ngữ lớn cỡ nhỏ, Hệ thống đa tác tử, Câu hỏi trắc nghiệm, Nhận dạng ký tự quang học, Phân loại Bloom, LLM-as-a-Judge

Abstract

Automatic multiple-choice question (MCQ) generation from educational documents is an essential yet challenging task, particularly when relying on proprietary large language models (LLMs) that incur high costs and require persistent internet connectivity. This thesis proposes the multi-agent architecture, to leverage small large language models (sLLMs) for high-quality MCQ generation from document images. The system deploys three parallel question-generation agents for different Bloom levels, together with a Classifier Agent for cognitive-level verification, a Student Agent for solvability checking, and a Fixer Agent for repairing answer options that fail quality control. A three-criteria evaluation framework (Solvability, Distractor Quality, Alignment) combined with the LLM-as-a-Judge methodology is developed for automated quality assessment. Additionally, a synthetic OCR data generation pipeline using Browser Engineering and Augraphy is introduced to create diverse evaluation datasets. Experimental results demonstrate that the multi-agent architecture significantly improves sLLM quality, boosting quality metrics by up to 93% for weaker models, all while maintaining a cost below \$0.50 per 100 passages. Notably, the best-performing sLLM (Gemma 3 with multi-agent orchestration) achieves competitive performance with proprietary LLMs at all Bloom levels. The Openlet web application is developed as a full-stack demonstration integrating the complete research architecture.

Keywords: Small Large Language Model, Multi-Agent System, Multiple-Choice Question, Optical Character Recognition, Bloom’s Taxonomy, LLM-as-a-Judge

Mục lục

Lời cảm ơn

Lời cam đoan i

Tóm tắt ii

Abstract iii

Mục lục iv

Danh sách hình vẽ vii

Danh sách bảng ix

Danh mục các từ viết tắt x

Chương 1 Mở đầu 1

1.1 Đặt vấn đề 1

1.2 Giải pháp đề xuất 2

1.3 Đóng góp của khóa luận 3

1.4 Cấu trúc khóa luận 4

Chương 2 Cơ sở lý thuyết & Các công trình liên quan 6

2.1 Phân loại Bloom 6

2.1.1 Cấp độ 1 - Truy xuất 6

2.1.2 Cấp độ 2 - Suy luận và tổng hợp 7

2.1.3 Cấp độ 3 - Lập luận phản biện 7

2.2 Hệ thống đa tác tử 8

2.2.1 Khái niệm hệ thống đa tác tử 8

2.2.2 Thư viện LangGraph 9

2.3 Phương pháp đánh giá bằng LLM 10

2.4	Các công trình liên quan	11
2.4.1	Sinh câu hỏi trắc nghiệm tự động bằng quy tắc	11
2.4.2	Sinh câu hỏi trắc nghiệm dựa trên mô hình ngôn ngữ lớn	12
2.5	Phân tích khoảng trống nghiên cứu	12
Chương 3 Phương pháp sinh câu hỏi trắc nghiệm bằng luồng đa tác tử		14
3.1	Kiến trúc hệ thống	14
3.2	Luồng xử lý đa tác tử	15
3.2.1	Tác tử <i>Sinh câu hỏi</i>	15
3.2.2	Tác tử <i>Phân loại</i>	16
3.2.3	Tác tử <i>Học sinh</i>	17
3.2.4	Tác tử <i>Hiệu chỉnh</i>	19
3.3	Một số kỹ thuật mở rộng	19
3.3.1	Suy luận chuỗi tư duy	19
3.3.2	Định dạng đầu ra của tác tử	20
Chương 4 Phương pháp đánh giá		22
4.1	Bộ dữ liệu	22
4.2	Quy trình tổng hợp bộ dữ liệu OCR	23
4.2.1	Sinh ảnh sạch từ văn bản	24
4.2.2	Tạo dữ liệu ảnh nhiễu mô phỏng thực tế	25
4.2.3	Kiểm thử nhận dạng văn bản	25
4.3	Thiết lập thực nghiệm	27
4.3.1	Các mô hình sử dụng	27
4.3.2	Cấu hình thực nghiệm	28
4.4	Phương pháp và Tiêu chí Đánh giá	29
4.4.1	Đánh giá chất lượng câu hỏi trắc nghiệm	29
4.4.2	Đánh giá hiệu năng nhận dạng văn bản	32
4.4.3	Các mô hình cơ sở	32
Chương 5 Kết quả		34
5.1	Kết quả sinh câu hỏi	34
5.1.1	Kết quả nhóm sLLM cơ sở	34
5.1.2	Kết quả nhóm sLLM kết hợp đa tác tử	35

5.1.3	Kết quả nhóm LLM thương mại	36
5.1.4	So sánh tổng hợp	36
5.2	Kết quả OCR	37
5.2.1	Kết quả trên ảnh sạch	38
5.2.2	Kết quả trên ảnh nhiễu	38
5.3	Phân tích chi phí	39
5.4	Thảo luận	42
Chương 6	Thiết kế và hiện thực ứng dụng Openlet	44
6.1	Giới thiệu ứng dụng và phạm vi sử dụng	44
6.2	Công nghệ sử dụng	45
6.2.1	Nhóm công nghệ giao diện	45
6.2.2	Nhóm công nghệ dịch vụ và lưu trữ	45
6.2.3	Nhóm công nghệ mô hình trí tuệ nhân tạo	45
6.3	Phân tích chức năng và tác nhân sử dụng	46
6.4	Thiết kế hệ thống	47
6.4.1	Kiến trúc tổng thể	47
6.4.2	Mô hình dữ liệu	48
6.4.3	Bảo mật và kiểm soát truy cập	50
6.5	Hiện thực các luồng chức năng cốt lõi	51
6.5.1	Luồng tạo bài kiểm tra tự động	51
6.5.2	Luồng truy cập, làm bài và nộp bài	53
6.6	Giao diện tiêu biểu của ứng dụng	54
6.6.1	Giao diện dành cho giáo viên	54
6.6.2	Giao diện dành cho người làm bài	55
Chương 7	Kết luận và Hướng phát triển	57
7.1	Tổng kết	57
7.2	Hạn chế	57
7.3	Hướng phát triển	58
	Tài liệu tham khảo	59

Danh sách hình vẽ

3.1	Kiến trúc hệ thống đa tác tử với hai vòng lặp kiểm soát chất lượng: vòng lặp phân loại cấp độ nhận thức và vòng lặp kiểm định tính giải được của câu hỏi	14
3.2	Luồng xử lý bên trong Tác tử Sinh câu hỏi: nhận văn bản nguồn và mẫu tham chiếu, sinh câu hỏi qua sLLM với prompt chuyên biệt theo cấp độ	16
3.3	Luồng xử lý bên trong Tác tử Phân loại: xáo trộn câu hỏi chống thiên lệch, phân loại cấp độ Bloom, và tách kết quả thành tập đạt chuẩn và tập sinh lại	17
3.4	Luồng xử lý bên trong Tác tử Học sinh: giải MCQ dạng nhiều lựa chọn đúng, phân luồng câu hỏi đạt chuẩn và câu hỏi cần hiệu chỉnh kèm lý do	18
3.5	Luồng xử lý bên trong Tác tử Hiệu chỉnh: nhận câu hỏi lỗi kèm lý do từ Tác tử Học sinh, chỉ sửa phương án và đáp án, đưa lại để tái kiểm định	19
3.6	Cấu trúc định dạng đầu ra chuẩn của tác tử <i>Sinh câu hỏi</i>	20
4.1	Sơ đồ quy trình tăng cường dữ liệu ảnh nhiễu: từ văn bản gốc qua kết xuất ảnh sạch, áp dụng nhiễu vật lý và quang học, đến ảnh giả lập tài liệu thực tế	24
4.2	Các mẫu ảnh tài liệu tổng hợp minh họa các loại nhiễu được áp dụng: nhiễu vật liệu (vết ố vàng, nhòe mực), nhiễu quang học (bóng đổ, nhiễu hạt), và biến dạng hình học (cong vênh trang)	26
5.1	So sánh chỉ số Acceptance trung bình giữa ba nhóm mô hình (sLLM cơ sở, sLLM kết hợp MAS, LLM thương mại) theo ba cấp độ nhận thức Bloom	37

5.2	So sánh CER và WER giữa điều kiện ảnh sạch và ảnh nhiễu cho bảy mô hình OCR, minh họa mức độ suy giảm hiệu suất khi chuyển sang điều kiện thực tế	40
5.3	Biểu đồ phân tán chi phí API (USD) so với Acceptance trung bình trên ba cấp độ nhận thức, xác định biên Pareto tối ưu giữa chất lượng và ngân sách	41
6.1	Biểu đồ ca sử dụng tổng quát của Openlet với hai tác nhân chính: giáo viên (quản lý vòng đời bài kiểm tra) và người làm bài (truy cập và nộp bài)	46
6.2	Kiến trúc ba tầng của Openlet: tầng giao diện (Next.js/React), tầng dịch vụ (Firebase), và tầng AI (OpenRouter kết nối các mô hình OCR/LLM)	47
6.3	Mô hình dữ liệu chính của Openlet gồm ba bảng: users (hồ sơ người dùng), quizzes (bài kiểm tra và câu hỏi), và attempts (lượt làm bài và kết quả)	49
6.4	Biểu đồ tuần tự của luồng tạo bài kiểm tra tự động: từ tiếp nhận tài liệu đầu vào, qua OCR và MAS sinh câu hỏi, đến cập nhật trạng thái theo thời gian thực	51
6.5	Biểu đồ tuần tự của luồng truy cập, làm bài và nộp bài: người làm bài nhận đề đã lọc đáp án, chấm điểm phía máy chủ, và nhận phản hồi theo mức cấu hình	53
6.6	Giao diện biên tập bài kiểm tra dành cho giáo viên: hiển thị câu hỏi đã sinh tự động với các công cụ chỉnh sửa nội dung, phương án và cấp độ nhận thức	55
6.7	Giao diện làm bài (trái) và trang kết quả (phải): người làm bài chọn đáp án với bộ đếm thời gian, sau đó nhận phản hồi theo mức hiển thị cấu hình bởi giáo viên	56

Danh sách bảng

4.1	Thống kê bộ dữ liệu RACE và mẫu thực nghiệm	23
4.2	Các mô hình sinh câu hỏi sử dụng trong thực nghiệm	27
4.3	Các mô hình OCR sử dụng trong thực nghiệm	28
5.1	Kết quả đánh giá nhóm sLLM cơ sở trên bộ dữ liệu RACE-High . .	34
5.2	Kết quả đánh giá nhóm sLLM tích hợp hệ thống đa tác tử trên bộ dữ liệu RACE-High	35
5.3	Kết quả đánh giá nhóm LLM thương mại trên bộ dữ liệu RACE-High	36
5.4	Kết quả OCR trên ảnh sạch	38
5.5	Kết quả OCR trên ảnh nhiễu	39
5.6	Chi phí sử dụng API cho 100 đoạn văn mẫu (USD)	40

Danh mục các từ viết tắt

Từ viết tắt	Cụm từ đầy đủ	Ý nghĩa / Mô tả
AMCQG	Automated Multiple-Choice Question Generation	Sinh câu hỏi trắc nghiệm tự động
API	Application Programming Interface	Giao diện lập trình ứng dụng
CAT	Computerized Adaptive Testing	Kiểm tra thích ứng trên máy tính
CER	Character Error Rate	Tỷ lệ lỗi ký tự
CNN	Convolutional Neural Network	Mạng nơ-ron tích chập
CoT	Chain-of-Thought	Chuỗi suy luận
LLM	Large Language Model	Mô hình ngôn ngữ lớn
LSTM	Long Short-Term Memory	Bộ nhớ ngắn hạn dài
MCQ	Multiple-Choice Question	Câu hỏi trắc nghiệm
NLP	Natural Language Processing	Xử lý ngôn ngữ tự nhiên
OCR	Optical Character Recognition	Nhận dạng ký tự quang học
PDF	Portable Document Format	Định dạng tài liệu di động
RAG	Retrieval-Augmented Generation	Sinh tăng cường truy xuất
RLHF	Reinforcement Learning from Human Feedback	Học tăng cường từ phản hồi con người
RNN	Recurrent Neural Network	Mạng nơ-ron hồi quy
sLLM	Small Large Language Model	Mô hình ngôn ngữ lớn cỡ nhỏ
MAS	Multi-Agent System	Hệ thống đa tác tử
VLM	Vision-Language Model	Mô hình thị giác - ngôn ngữ
WER	Word Error Rate	Tỷ lệ lỗi từ

Chương 1

Mở đầu

Chương này trình bày về bối cảnh, lý do chọn đề tài nghiên cứu cũng như phân tích các thách thức hiện tại trong bài toán số hóa học liệu và tự động sinh câu hỏi trắc nghiệm. Từ đó, chương đưa ra giải pháp đề xuất dựa trên các mô hình ngôn ngữ lớn cỡ nhỏ và kiến trúc đa tác tử, đồng thời xác định rõ mục tiêu, phạm vi nghiên cứu và cấu trúc tổng thể của khóa luận.

1.1 Đặt vấn đề

Trong bối cảnh kỷ nguyên số đang định hình lại mọi lĩnh vực của đời sống, giáo dục cũng đang trải qua một cuộc chuyển đổi số mạnh mẽ. Một trong những yêu cầu cấp thiết đối với các cơ sở giáo dục hiện nay là việc số hóa các tài liệu giảng dạy, kho đề thi, và học liệu từ dạng tài liệu giấy hoặc ảnh chụp sang các định dạng dữ liệu có cấu trúc. Việc này không chỉ giúp lưu trữ và chia sẻ dễ dàng trên các hệ thống quản lý học tập mà còn mở ra cơ hội cá nhân hóa lộ trình học tập cho từng học sinh. Đồng thời, công tác kiểm tra và đánh giá là một trụ cột không thể thiếu trong giáo dục, trong đó bài thi trắc nghiệm nhiều lựa chọn đóng vai trò đặc biệt quan trọng nhờ tính khách quan, khả năng bao quát kiến thức rộng và dễ dàng chấm điểm tự động. Tuy nhiên, việc thiết kế một bộ câu hỏi trắc nghiệm chất lượng cao, đặc biệt là các câu hỏi đòi hỏi tư duy bậc cao theo thang phân loại **Bloom** [4], tiêu tốn rất nhiều thời gian và chất xám của giáo viên.

Quá trình tự động hóa việc chuyển đổi học liệu thành câu hỏi trắc nghiệm hiện nay đang vấp phải hai rào cản lớn về mặt kỹ thuật:

- **Thách thức trong việc số hóa tài liệu từ ảnh chụp:** Các học liệu thực tế thường ở dạng sách in, giấy tờ hoặc ảnh chụp bởi điện thoại với chất lượng quang học thấp (mờ, nhòe, nhiễu hạt, ánh sáng không đều). Các công cụ nhận dạng ký tự quang học (OCR - Optical Character Recognition) truyền thống như Tesseract [28] mặc dù phổ biến nhưng thường gặp khó khăn nghiêm trọng trong việc xử lý các tài liệu chứa cấu trúc phức tạp (bảng biểu, công thức

toán học, biểu đồ) hoặc bị nhiễu. Hậu quả là văn bản được trích xuất thường xuyên bị vỡ cấu trúc đoạn văn, sai lệch ngữ nghĩa, khiến cho đầu vào của giai đoạn sinh câu hỏi tiếp theo trở nên vô nghĩa.

- **Thách thức trong việc kiểm soát chất lượng sinh câu hỏi:** Các phương pháp sinh câu hỏi truyền thống thường dựa trên các bộ quy tắc cú pháp ngữ pháp học (rule-based NLP) [13], dẫn đến việc hệ thống chỉ có thể sinh ra các câu hỏi ở mức độ nhận thức thấp, chủ yếu là ghi nhớ sự kiện, rất rập khuôn và thiếu tự nhiên. Gần đây, sự xuất hiện của các mô hình ngôn ngữ lớn (LLM - Large Language Model) và mô hình thị giác - ngôn ngữ (VLM - Vision-Language Model) khổng lồ như GPT [27], Gemini [29] hay Claude [3] đã giải quyết được cả hai bài toán trên với độ chính xác rất cao. Tuy nhiên, việc ứng dụng các mô hình này trong môi trường giáo dục vấp phải rào cản chí mạng: chi phí gọi giao diện lập trình ứng dụng (API - Application Programming Interface) vô cùng đắt đỏ, phụ thuộc hoàn toàn vào kết nối internet ổn định và tiềm ẩn rủi ro lớn về bảo mật dữ liệu. Thêm vào đó, ngay cả với LLM lớn, việc sinh ra các câu hỏi tư duy sâu (Suy luận, Phân tích) với phương án nhiều có tính chẩn đoán cao chỉ qua một lần yêu cầu vẫn thường dẫn đến lỗi logic hoặc “ảo giác” (hallucination).

Từ những rào cản đó, một bài toán cấp thiết được đặt ra: Làm thế nào để xây dựng một quy trình số hóa và sinh câu hỏi tự động, đảm bảo chất lượng sư phạm cao, có khả năng xử lý hình ảnh nhiễu, nhưng vẫn tối ưu hóa được chi phí và có thể triển khai trên các hạ tầng phần cứng nội bộ, đảm bảo tính bảo mật?

1.2 Giải pháp đề xuất

Để giải quyết bài toán trên, khóa luận này đề xuất một giải pháp đột phá và toàn diện: Xây dựng một luồng xử lý khép kín ứng dụng các mô hình ngôn ngữ lớn cỡ nhỏ mã nguồn mở (sLLM - Small Large Language Model, với quy mô tham số dưới 15 tỷ, có thể chạy trên máy tính cá nhân hoặc máy chủ nội bộ vừa phải) kết hợp với kiến trúc hệ thống đa tác tử (MAS - Multi-Agent System).

Quy trình giải pháp được chia thành hai giai đoạn cốt lõi:

1. **Giai đoạn trích xuất văn bản bằng VLM nhỏ gọn:** Thay vì sử dụng

OCR truyền thống, hệ thống tích hợp các VLM chuyên dụng cỡ nhỏ (như DeepSeek OCR-2, MinerU, PaddleOCR-VL). Các mô hình này nhận đầu vào là ảnh chụp tài liệu chứa yếu tố nhiễu, sau đó tận dụng khả năng hiểu cấu trúc tài liệu thông minh để trích xuất văn bản chuẩn xác, bảo toàn định dạng đoạn văn, giúp tạo ra văn bản chính xác cho giai đoạn sinh câu hỏi.

2. **Giai đoạn sinh câu hỏi trắc nghiệm bằng kiến trúc đa tác tử:** Thay vì dùng một câu lệnh duy nhất ép mô hình sLLM (như Gemma 3, Qwen 3, Phi-4) phải làm tất cả mọi việc, nghiên cứu đề xuất phân rã quy trình tư duy của giáo viên thành các vai trò riêng biệt. Hệ thống **LangGraph** [15] được sử dụng để điều phối một mạng lưới các tác tử:

- **Tác tử Phân loại:** Định hướng và đảm bảo câu hỏi chuẩn bị sinh ra tuân thủ đúng mức độ nhận thức trong thang Bloom (Nhớ, Hiểu, Phân tích).
- **Tác tử Sinh câu hỏi:** Dựa trên ngữ cảnh và phân loại đã định, tạo ra câu hỏi thô cùng bộ đáp án (bao gồm đáp án đúng và các phương án nhiễu).
- **Tác tử Học sinh:** Đóng vai trò phản biện. Tác tử này cố gắng “giải” câu hỏi vừa được sinh ra mà không xem đáp án trước. Quá trình này giúp phát hiện ra các câu hỏi bị đa nghĩa, bẫy logic bất hợp lý, hoặc phương án nhiễu quá dễ nhận biết.
- **Tác tử Hiệu chỉnh:** Dựa trên phản hồi từ Tác tử Học sinh, tác tử này tiến hành sửa đổi, tinh chỉnh ngôn từ và logic để câu hỏi đạt chuẩn sư phạm cao nhất.

Nhờ sự giám sát chéo và quy trình tinh chỉnh lặp đi lặp lại của MAS, các sLLM tuy có lượng tham số nhỏ nhưng lại có thể sản sinh ra chất lượng nội dung tiệm cận với các LLM khổng lồ thương mại, đồng thời giải quyết triệt để bài toán bảo mật và chi phí vận hành.

1.3 Đóng góp của khóa luận

Thông qua việc thiết kế, triển khai và đánh giá giải pháp, khóa luận mang đến 4 đóng góp chính mang ý nghĩa cả về mặt học thuật lẫn ứng dụng thực tiễn:

- **Đề xuất và thiết kế kiến trúc hệ thống đa tác tử chuyên biệt cho giáo dục:** Khóa luận lần đầu tiên hệ thống hóa và áp dụng thành công kiến

trúc MAS dựa trên sLLM cho bài toán sinh câu hỏi trắc nghiệm đa cấp độ nhận thức theo thang Bloom. Cơ chế tự kiểm duyệt qua vai trò “Giáo viên - Học sinh” đã bù đắp xuất sắc giới hạn suy luận nội tại của các mô hình ngôn ngữ cỡ nhỏ.

- **Đánh giá thực nghiệm quy mô và toàn diện:** Khóa luận thiết kế một quy trình sinh ảnh tài liệu giả lập (từ tập dữ liệu RACE [14]) để mô phỏng chính xác các điều kiện nhiễu trong thực tế. Dựa trên đó, nghiên cứu tiến hành đánh giá và so sánh chi tiết hiệu năng của 12 mô hình khác nhau (cả VLM, sLLM mã nguồn mở và LLM thương mại) trên 2 bài toán OCR và sinh câu hỏi, cung cấp những minh chứng thực nghiệm rõ ràng về tiềm năng thay thế của sLLM.
- **Phát triển khung đánh giá chất lượng sư phạm:** Khắc phục điểm yếu của các độ đo n-gram truyền thống (như BLEU, ROUGE), nghiên cứu thiết lập một quy trình đánh giá tự động đa chiều dựa trên 3 tiêu chí sư phạm thiết yếu: Tính giải được (Solvability), Độ bám sát phân loại nhận thức (Alignment), và Chất lượng phương án nhiễu (Distractor Quality).
- **Hiện thực hóa giải pháp qua ứng dụng Web Openlet:** Khóa luận không dừng lại ở mức độ lý thuyết mà tiến hành xây dựng Openlet, một nền tảng ứng dụng Web trực quan, cho phép người dùng cuối (giáo viên, học sinh) tải ảnh tài liệu lên, hệ thống sẽ tự động trích xuất và sinh ra bài kiểm tra trực tuyến. Điều này chứng minh rõ ràng tính khả thi trong việc triển khai giải pháp vào môi trường giáo dục số thực tế.

1.4 Cấu trúc khóa luận

Bên cạnh nội dung báo cáo, toàn bộ mã nguồn và kết quả thực nghiệm cũng được công bố dưới dạng mã nguồn mở¹. Để trình bày một cách có hệ thống quá trình nghiên cứu, thiết kế và đánh giá, nội dung khóa luận được tổ chức thành 7 chương với bố cục như sau:

- **Chương 1 - Mở đầu:** Đặt vấn đề, phân tích các rào cản trong các phương

¹<https://github.com/qninhdt/openlet-research>

pháp hiện tại, từ đó đề xuất giải pháp ứng dụng sLLM kết hợp MAS, đồng thời tóm tắt các đóng góp chính của nghiên cứu.

- **Chương 2 - Cơ sở lý thuyết & Các công trình liên quan:** Cung cấp nền tảng kiến thức về thang phân loại nhận thức Bloom và nền tảng của MAS. Phần cuối chương tập hợp các nghiên cứu liên quan để chỉ ra khoảng trống nghiên cứu hiện tại.
- **Chương 3 - Phương pháp đề xuất:** Trình bày chuyên sâu về kiến trúc MAS đề xuất và thiết kế của từng tác tử trong quy trình sinh câu hỏi.
- **Chương 4 - Phương pháp đánh giá:** Thiết lập môi trường thực nghiệm, mô tả bộ dữ liệu đầu vào, chiến lược tạo dữ liệu ảnh nhiễu, các tham số cấu hình mô hình, và thiết lập các mô hình cơ sở để đối chứng kết quả.
- **Chương 5 - Kết quả:** Phân tích số liệu thực nghiệm. So sánh chất lượng OCR, năng lực sinh câu hỏi đa cấp độ của sLLM so với LLM thương mại, và phân tích bài toán hiệu quả chi phí.
- **Chương 6 - Thiết kế và hiện thực ứng dụng Openlet:** Mô tả quá trình kỹ thuật hóa từ lý thuyết thành sản phẩm thực tế, bao gồm sơ đồ ca sử dụng, kiến trúc phần mềm, cũng như giới thiệu các tính năng cốt lõi của nền tảng web Openlet.
- **Chương 7 - Kết luận và Hướng phát triển:** Tổng kết các kết quả quan trọng nhất mà nghiên cứu đạt được, thành thật nhìn nhận những hạn chế còn tồn tại và đề xuất các định hướng phát triển cho hệ thống trong tương lai.

Chương 2

Cơ sở lý thuyết & Các công trình liên quan

Trong chương này, chúng ta sẽ đi sâu vào hai nền tảng lý thuyết cốt lõi của nghiên cứu này: Thang phân loại nhận thức Bloom và Hệ thống đa tác tử. Phần cuối chương sẽ tổng hợp các công trình liên quan để chỉ ra khoảng trống nghiên cứu hiện tại mà khóa luận này hướng tới giải quyết.

2.1 Phân loại Bloom

Để các hệ thống sinh câu hỏi trắc nghiệm tự động có thể vừa đảm bảo các mục tiêu sư phạm vừa không biến thành một cỗ máy tạo câu hỏi ngẫu nhiên, việc thiết kế độ khó cần dựa trên một thang đo đã được kiểm chứng bởi khoa học. Nghiên cứu này tập trung vào khung nhận thức được công nhận rộng rãi nhất trong lĩnh vực giáo dục là **phân loại Bloom**. Thang đo này được nhà tâm lý học và giáo dục *Benjamin Bloom* đề xuất lần đầu vào năm 1956 [4] và được tinh chỉnh bởi *Anderson* và *Krathwohl* vào năm 2001 [2]. Trong nghiên cứu này, nhằm tối ưu hóa và phù hợp với hình thức thi câu hỏi trắc nghiệm đa lựa chọn (MCQ - Multiple-Choice Question), sáu cấp độ ban đầu của thang đo Bloom được phân loại lại thành ba cấp độ cốt lõi là Truy xuất, Suy luận và tổng hợp, và Lập luận phản biện.

2.1.1 Cấp độ 1 - Truy xuất

Cấp độ Truy xuất tương ứng với bậc Nhớ trong thang đo Bloom sửa đổi. Cấp độ này là nền tảng của sự nhận thức, tập trung chủ yếu vào khả năng ghi nhớ cơ học và truy xuất thông tin dài hạn, hoặc nhận dạng một đối tượng, sự kiện được trình bày rõ ràng trong một đoạn văn bản thuộc bài đọc. Quá trình này thường được gọi là nhận thức nông.

Trong quá trình tinh chỉnh các chỉ dẫn cho LLMs sinh câu hỏi ở cấp độ Truy xuất, hệ thống được cài đặt để tập trung vào các chi tiết cụ thể, định nghĩa thuật ngữ

và nhận dạng các sự kiện. Đặc thù của MCQ ở dạng này thường xoay quanh các câu hỏi “Ai?”, “Cái gì?”, “Ở đâu?” và “Khi nào?”. Đối với bài toán này, LLMs thể hiện vô cùng xuất sắc trong việc tự động tạo ra các câu hỏi, bởi vì quy trình này cơ bản chỉ đòi hỏi khả năng đối sánh chuỗi và trích xuất các thực thể từ văn bản mà không cần hiểu sâu hay xây dựng bất kỳ chuỗi suy luận logic nào. Không chỉ vậy, các phương án nhiễu ở mức độ này cũng tương đối dễ xây dựng; chúng thường là các thực thể hoặc dữ liệu xuất hiện trong văn bản gốc để tạo ra sự khó khăn cho người đọc.

2.1.2 Cấp độ 2 - Suy luận và tổng hợp

Cấp bậc thứ hai bao gồm hai kỹ năng Suy luận và Tổng hợp, đại diện cho các bậc Hiểu, Áp dụng và Sáng tạo trong phân loại Bloom. Ở cấp độ này, thay vì được hỏi các câu hỏi có thể truy xuất trực tiếp từ bề mặt của văn bản, người học sẽ buộc phải xử lý thông tin đã học và ứng dụng lý thuyết đó vào một ngữ cảnh hoàn toàn mới hoặc một tình huống giả định.

Trong bối cảnh bài toán MCQ, Suy luận yêu cầu người học phải có khả năng liên kết các mệnh đề nằm rải rác trong bài đọc để rút ra các kết quả, nhận định không được tác giả viết ra một cách trực tiếp. Đối với Tổng hợp, cấp độ này yêu cầu người học phải chắt lọc, sắp xếp lại các yếu tố riêng lẻ thành một góc nhìn bao quát hoặc dự đoán một kết quả tương lai. Bài toán này đòi hỏi cần có các chỉ dẫn cụ thể và hợp lý cho LLMs. Ngoài ra, việc tạo ra các đáp án nhiễu ở cấp độ này cũng là một thách thức kỹ thuật đối với AI: các đáp án sai phải được thiết kế dựa vào sự suy diễn sai lầm về mặt logic hoặc dựa vào việc người đọc hiểu sai một quy luật nào đó để từ đó có thể tạo ra các lựa chọn bẫy hợp lý về mặt trực giác.

2.1.3 Cấp độ 3 - Lập luận phản biện

Cấp độ cuối cùng của nghiên cứu này là Cấp độ Lập luận phản biện, tương ứng với bậc Phân tích và Đánh giá. Trọng tâm của cấp độ nhận thức này là khả năng chia nhỏ một lượng thông tin đồ sộ, phức tạp thành các thành phần nhỏ hơn nhằm phân tích, nhận diện các giả định ngầm và đưa ra những nhận xét, quan điểm khách quan dựa trên sự đối chiếu với các bằng chứng thực tiễn.

Các câu hỏi trắc nghiệm ở cấp độ này thường yêu cầu người học tưởng tượng ra

các tình huống kiểm tra nguy hiểm logic, yêu cầu họ phân biệt rõ ràng giữa một sự thật khách quan và một quan điểm chủ quan, hoặc được yêu cầu đánh giá độ chính xác, hợp lý của một lập luận khoa học. Thách thức lớn nhất mà các LLM phải đối mặt ở cấp độ này là việc sinh ra các câu hỏi chất lượng cao chỉ có duy nhất một đáp án đúng. Quan trọng hơn cả, phương án nhiều ở mức độ này không thể chỉ là những thông tin sai lệch thông thường, mà phải là các lập luận, lý lẽ nhìn bề mặt có vẻ cực kỳ thuyết phục nhưng lại chứa các lỗ hổng về mặt logic, từ đó buộc người học phải sử dụng tư duy phản biện cao mới có thể nhận diện được.

2.2 Hệ thống đa tác tử

2.2.1 Khái niệm hệ thống đa tác tử

Hệ thống đa tác tử đại diện cho một thể hệ kiến trúc trí tuệ nhân tạo hiện đại, cho phép giải quyết các bài toán phức tạp mà một LLM duy nhất khó có thể xử lý hiệu quả. Cách tiếp cận này chia nhỏ bài toán thành các nhiệm vụ hẹp hơn và giao cho một mạng lưới gồm nhiều tác tử chuyên biệt có khả năng tương tác qua lại. Một sai lầm phổ biến của nhiều hệ thống là yêu cầu một LLM giải quyết trọn vẹn một bài toán đòi hỏi quy trình phức tạp về logic và nghiệp vụ, khiến kết quả đầu ra kém chất lượng và khó sử dụng. Nguyên nhân chính là mô hình bị giao quá nhiều vai trò cùng lúc như đọc hiểu văn bản dài, sinh câu hỏi và tạo đáp án nhiều trong cùng một lần gọi.

Hệ thống đa tác tử giải quyết những vấn đề này bằng cách mô phỏng cách con người phân công vai trò và phối hợp xử lý công việc, đồng thời dựa trên triết lý của **ReAct** [37]. Thay vì chỉ trả lời một cách thụ động, khung ReAct buộc LLM thực hiện một vòng lặp nội tại gồm suy nghĩ, hành động, quan sát kết quả và điều chỉnh suy nghĩ. Trong hệ thống đa tác tử, mỗi tác tử được trang bị một chỉ thị hẹp nhưng có mức chuyên môn hóa cao, cho phép tập trung tài nguyên nhận thức vào một vấn đề duy nhất. Ví dụ, Tác tử Phân tích chỉ chịu trách nhiệm lọc và cấu trúc hóa văn bản thô; sau đó kết quả được chuyển cho các Tác tử Sinh câu hỏi để tạo ra bộ câu hỏi ban đầu, rồi tiếp tục được chuyển cho các Tác tử Đánh giá và Sửa lỗi để thực hiện vòng cải thiện chất lượng. Sự chuyên môn hóa này không chỉ giảm thiểu ảo giác mà còn giúp cải thiện chất lượng nội dung sinh ra, giảm lỗi và duy trì độ ổn định cho toàn hệ thống.

2.2.2 Thư viện LangGraph

Để cài đặt và triển khai hệ thống đa tác tử một cách đúng đắn, nghiên cứu này sử dụng **LangGraph** [15], một thư viện mã nguồn mở chuyên dụng được xây dựng trên nền tảng của hệ sinh thái **LangChain** [5]. Khác với triết lý xử lý tuần tự tuyến tính của LangChain, LangGraph được thiết kế riêng cho việc tạo ra các luồng tác tử thông qua đồ thị có hướng kèm theo một đối tượng trạng thái dùng để lưu trữ dữ liệu khi hệ thống duyệt qua từng nút trong đồ thị.

Ba thành phần chính của một hệ thống đa tác tử khi được triển khai bằng LangGraph bao gồm:

1. **Trạng thái:** Là một cấu trúc dữ liệu trung tâm, hoạt động như bộ nhớ chung để duy trì ngữ cảnh cho toàn bộ hệ thống, ví dụ như các ý chính đã phân tích từ văn bản gốc, các câu hỏi đã tạo và lỗi của từng câu hỏi.
2. **Nút:** Đây là các hàm Python có nhiệm vụ xử lý logic cho từng tác tử (Nút sinh câu hỏi cho cấp độ 3, Nút thực hiện đánh giá câu hỏi cấp độ 2, ...). Nút này sẽ nhận được Trạng thái hiện tại, gửi lệnh đến LLM để xử lý và sau đó cập nhật Trạng thái mới với kết quả vừa tạo ra.
3. **Cạnh:** Thành phần này có nhiệm vụ xác định hướng di chuyển của luồng dữ liệu giữa các nút trong đồ thị. LangGraph cho phép hệ thống tự đưa ra quyết định rẽ nhánh, kích hoạt cơ chế thực thi song song, hoặc kết thúc luồng hoạt động dựa trên các tiêu chí xuất hiện trong thời gian thực.

Khả năng hỗ trợ cấu trúc đồ thị của LangGraph cho phép triển khai các hệ thống trí tuệ nhân tạo tự định hướng. Nếu một tác tử Đánh giá phát hiện câu hỏi sinh ra có chứa phương án nhiễu vô lý hay nhiều đáp án đúng, hệ thống có thể yêu cầu một Tác tử Sửa lỗi xử lý vấn đề rồi đưa kết quả đó quay lại để đánh giá tiếp. Chu trình này chỉ dừng khi hệ thống tạo ra kết quả đạt chất lượng mong muốn. Ngoài ra, LangGraph còn được trang bị bộ nhớ chốt chặn giúp hệ thống có thể phục hồi trạng thái khi gặp các vấn đề như lỗi giao diện lập trình ứng dụng.

2.3 Phương pháp đánh giá bằng LLM

Để khắc phục các hạn chế về chi phí và khả năng mở rộng của đánh giá bởi con người, cộng đồng nghiên cứu đã triển khai phương pháp sử dụng mô hình ngôn ngữ lớn làm giám khảo tự động (LLM-as-a-Judge). Ý tưởng cốt lõi của phương pháp này là tận dụng năng lực hiểu ngôn ngữ và suy luận của các LLM thương mại mạnh mẽ để thay thế vai trò của con người trong các quy trình thẩm định chất lượng.

Một trong những khung đánh giá tiêu biểu nhất là **G-Eval** [17], được đề xuất bởi Liu và cộng sự vào năm 2023. G-Eval thiết kế một quy trình đánh giá có cấu trúc, trong đó LLM giám khảo nhận đầu vào gồm ba thành phần: nội dung nguồn, nội dung cần đánh giá và bộ tiêu chuẩn đánh giá chi tiết. Điểm cải tiến then chốt của G-Eval so với các phương pháp đánh giá tự động trước đó là yêu cầu bắt buộc tích hợp kỹ thuật chuỗi suy nghĩ (CoT - Chain-of-Thought [34]): trước khi đưa ra điểm số, LLM giám khảo phải trình bày chuỗi lập luận logic giải thích quá trình đánh giá. Cơ chế này giúp tăng tính minh bạch và cho phép con người kiểm tra tính hợp lý của kết quả đánh giá.

Phương pháp LLM-as-a-Judge mang lại ba lợi thế quan trọng. Thứ nhất, khả năng mở rộng vượt trội: hệ thống có thể đánh giá hàng nghìn câu hỏi trong thời gian ngắn với chi phí thấp hơn nhiều so với đánh giá bởi con người. Thứ hai, tính nhất quán cao: cùng một bộ tiêu chuẩn đánh giá, LLM giám khảo luôn đưa ra kết quả ổn định và có thể tái lập, loại bỏ yếu tố chủ quan giữa các người đánh giá khác nhau. Thứ ba, tính linh hoạt: bộ tiêu chuẩn đánh giá có thể được điều chỉnh dễ dàng cho các miền kiến thức và ngữ cảnh giáo dục khác nhau mà không cần đào tạo lại đội ngũ chuyên gia.

Tuy nhiên, phương pháp này cũng tồn tại các hạn chế cần lưu ý. LLM giám khảo có thể mắc các thiên lệch hệ thống như ưu tiên các câu trả lời dài hơn, thiên vị đầu ra của chính mình, hoặc thiếu nhạy cảm với các lỗi sơ phạm tinh tế mà chuyên gia giáo dục có thể nhận ra ngay lập tức. Do đó, kết quả từ LLM-as-a-Judge nên được xem như một công cụ sàng lọc hiệu quả, nhưng vẫn cần được đối chiếu với đánh giá của con người để xác nhận độ tin cậy.

2.4 Các công trình liên quan

Phần này trình bày tổng quan các hướng nghiên cứu liên quan đến bài toán sinh câu hỏi trắc nghiệm tự động từ ảnh tài liệu, tập trung vào hai nhóm chính: phương pháp sinh câu hỏi truyền thống và sinh câu hỏi dựa trên mô hình ngôn ngữ lớn.

2.4.1 Sinh câu hỏi trắc nghiệm tự động bằng quy tắc

Các nghiên cứu tiên phong về sinh câu hỏi trắc nghiệm tự động chủ yếu dựa trên các kỹ thuật xử lý ngôn ngữ tự nhiên (NLP - Natural Language Processing) truyền thống và các quy tắc ngôn ngữ học được thiết kế thủ công. *Heilman* và *Smith* [13] đề xuất phương pháp tiếp cận dựa trên xếp hạng thống kê cho bài toán sinh câu hỏi, trong đó hệ thống sinh ra một lượng lớn câu hỏi ứng viên từ văn bản đầu vào thông qua các phép biến đổi cú pháp, sau đó sử dụng mô hình xếp hạng thống kê để lọc và chọn ra các câu hỏi có chất lượng cao nhất. Cách tiếp cận này đã trở thành một khuôn mẫu phổ biến trong lĩnh vực sinh câu hỏi tự động ở giai đoạn trước khi học sâu được áp dụng rộng rãi.

Trong một hướng tiếp cận khác, *Mitkov* và cộng sự [19] phát triển một hệ thống sinh câu hỏi trắc nghiệm hỗ trợ bằng máy tính, tích hợp nhiều kỹ thuật NLP bao gồm nhận dạng thực thể có tên, gán nhãn từ loại và phân tích cú pháp. Hệ thống này tự động xác định các khái niệm quan trọng trong văn bản nguồn, sau đó áp dụng các quy tắc biến đổi ngôn ngữ linh hoạt để tạo thành câu hỏi và các phương án nhiễu tương ứng.

Các phương pháp truyền thống có ưu điểm nổi bật là khả năng kiểm soát cao về mặt ngữ pháp và cấu trúc câu hỏi đầu ra, đồng thời cho phép người thiết kế can thiệp trực tiếp vào quy trình sinh câu hỏi. Tuy nhiên, các phương pháp này tồn tại nhiều hạn chế căn bản: (i) các câu hỏi được sinh ra chủ yếu nằm ở mức nhận thức thấp theo thang Bloom [4], tập trung vào khả năng ghi nhớ và tái hiện thông tin bề mặt; (ii) chất lượng của các phương án nhiễu thường không đạt yêu cầu sư phạm do thiếu khả năng suy luận ngữ nghĩa sâu; (iii) hệ thống đòi hỏi thiết kế riêng bộ quy tắc ngôn ngữ học cho từng ngôn ngữ và từng lĩnh vực chuyên môn, gây khó khăn lớn trong việc mở rộng phạm vi ứng dụng. Đặc biệt, các phương pháp này không có khả năng sinh các câu hỏi đòi hỏi suy luận hay phản biện, vốn là các mức nhận thức bậc cao cần thiết trong đánh giá giáo dục hiện đại [2].

2.4.2 Sinh câu hỏi trắc nghiệm dựa trên mô hình ngôn ngữ lớn

Sự ra đời của các LLM đã tạo ra một bước chuyển quan trọng trong lĩnh vực sinh câu hỏi tự động, từ các hệ thống dựa trên quy tắc sang các phương pháp dựa trên khả năng sinh ngôn ngữ tự nhiên của mô hình. Thay vì thiết kế thủ công các quy tắc biến đổi cú pháp, các nghiên cứu từ năm 2022 trở đi đã tận dụng khả năng hiểu ngữ cảnh sâu và sinh văn bản mạch lạc của LLM để tạo ra câu hỏi trắc nghiệm (MCQ - Multiple-Choice Question) trực tiếp từ đoạn văn bản nguồn.

Elkins và cộng sự [7] thực hiện đánh giá có hệ thống về chất lượng câu hỏi trong lĩnh vực giáo dục được sinh bởi các LLM. Kết quả nghiên cứu cho thấy các mô hình này có khả năng tạo ra các câu hỏi đạt yêu cầu sử dụng trong thực tế giảng dạy, tuy nhiên vẫn gặp khó khăn đáng kể trong việc sinh các câu hỏi đòi hỏi tư duy bậc cao theo phân loại Bloom. Nghiên cứu này nhấn mạnh rằng khoảng cách giữa câu hỏi ghi nhớ đơn thuần và câu hỏi phân tích, đánh giá vẫn là thách thức lớn đối với phương pháp sinh câu hỏi chỉ dùng một lần gọi mô hình.

Bên cạnh khả năng sinh câu hỏi, việc sử dụng LLM để tự động đánh giá chất lượng câu hỏi cũng là một hướng đi đáng chú ý. *Moore* và cộng sự [21] đã tiến hành nghiên cứu sử dụng GPT-4 để đánh giá và phát hiện các lỗi biên soạn trong các câu hỏi trắc nghiệm do sinh viên tạo ra. Khi đối chiếu với kết quả đánh giá của chuyên gia giáo dục, nghiên cứu phát hiện ra rằng độ chính xác của GPT-4 (79%) vẫn kém hơn đáng kể so với một hệ thống đánh giá bằng tập luật truyền thống (91%). Hơn nữa, GPT-4 có xu hướng “bắt lỗi” quá khắt khe, thường xuyên nhận diện sai các lỗi sơ phạm không thực sự tồn tại. Phát hiện này củng cố cho nhận định rằng: mặc dù LLM rất mạnh mẽ trong việc xử lý ngôn ngữ, nhưng nếu chỉ sử dụng trực tiếp một LLM đơn lẻ làm giám khảo kiểm định mà thiếu đi các cơ chế kiểm soát chéo, hệ thống sẽ rất dễ đưa ra các đánh giá sai lệch về mặt sơ phạm.

2.5 Phân tích khoảng trống nghiên cứu

Dựa trên quá trình tổng quan các công trình liên quan đã trình bày, nghiên cứu rút ra ba khoảng trống chính còn tồn tại trong lĩnh vực sinh câu hỏi trắc nghiệm tự động, thể hiện rõ những hạn chế của các phương pháp hiện hành:

- Khả năng kiểm soát sơ phạm và độ chính xác thấp ở tư duy bậc

cao: Các phương pháp sinh câu hỏi truyền thống dựa trên quy tắc cấu trúc (như phân tích cú pháp NLP, từ điển WordNet) tỏ ra cứng nhắc, chủ yếu tạo ra được những câu hỏi ở mức độ nhận thức thấp. Ngược lại, việc sử dụng trực tiếp một mô hình ngôn ngữ lớn để sinh câu hỏi thường dẫn đến tình trạng “ảo giác” (hallucination) hoặc không thể thiết kế các phương án nhiều có tính chẩn đoán logic. Các mô hình đơn lẻ thường thất bại trong việc phân định ranh giới giữa các cấp độ nhận thức cao như Suy luận và Lập luận phản biện.

- **Tốn kém tài nguyên và rào cản triển khai thực tế:** Các hệ thống tiên tiến hiện nay phụ thuộc quá nhiều vào các mô hình thương mại khổng lồ. Dù mang lại chất lượng câu hỏi tương đối tốt, các mô hình này lại đi kèm với chi phí gọi API đắt đỏ, độ trễ hệ thống cao và đòi hỏi kết nối mạng liên tục. Đối với môi trường giáo dục có ngân sách hạn hẹp hoặc các tổ chức yêu cầu bảo mật dữ liệu đề thi nghiêm ngặt, việc phụ thuộc vào nền tảng bên ngoài là một rào cản rất lớn.
- **Sự phân mảnh trong quy trình số hóa toàn diện:** Ít có hệ thống nào giải quyết trọn vẹn luồng công việc từ lúc nhận một bức ảnh tài liệu thô (có nhiều yếu tố nhiễu vật lý như mờ, nhòe) đến khi tạo ra bộ câu hỏi hoàn chỉnh. Các công cụ nhận dạng ký tự (OCR) truyền thống thường phá vỡ cấu trúc văn bản, làm mất ngữ cảnh trước khi đưa vào mô hình sinh câu hỏi, khiến luồng dữ liệu trở nên kém hiệu quả.

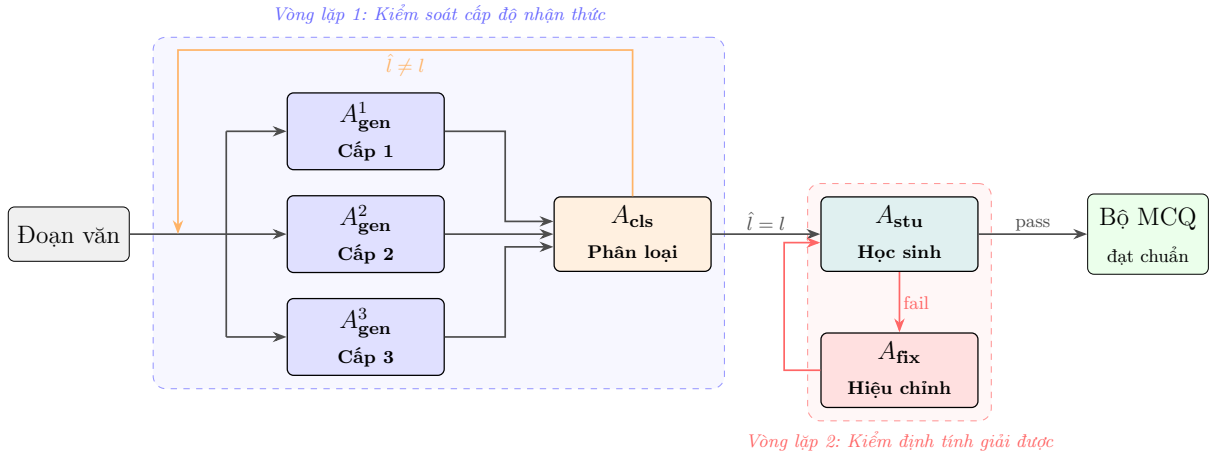
Sự cần thiết của giải pháp đề xuất: Từ những hạn chế cốt lõi trên - sự đắt đỏ và thiếu tính bảo mật của LLM thương mại, sự kém hiệu quả của các quy tắc NLP cũ, cũng như sự thiếu hụt cơ chế tự kiểm duyệt cho các câu hỏi tư duy bậc cao - giải pháp của nghiên cứu này ra đời. Hệ thống sử dụng **các mô hình cỡ nhỏ mã nguồn mở** nhằm giải quyết triệt để bài toán tài nguyên và bảo mật. Đồng thời, việc ứng dụng kiến trúc **đa tác tử** đóng vai trò như một cơ chế tự giám sát, mô phỏng quá trình tư duy độc lập, qua đó tối ưu hóa độ chính xác và đảm bảo chất lượng sư phạm cho các câu hỏi trắc nghiệm phức tạp mà không cần sử dụng đến các hệ thống quá khổng lồ.

Chương 3

Phương pháp sinh câu hỏi trắc nghiệm bằng luồng đa tác tử

3.1 Kiến trúc hệ thống

Chất lượng của câu hỏi trắc nghiệm được sinh ra phụ thuộc trực tiếp vào cách thức thông tin được tổ chức và truyền đạt đến mô hình ngôn ngữ tại mỗi bước trong pipeline. Phần này trình bày thiết kế đối tượng trạng thái chia sẻ, luồng xử lý giữa các tác tử, vai trò và prompt của từng tác tử, cùng các kỹ thuật đảm bảo tính ổn định của hệ thống.



Hình 3.1: Kiến trúc hệ thống đa tác tử với hai vòng lặp kiểm soát chất lượng: vòng lặp phân loại cấp độ nhận thức và vòng lặp kiểm định tính giải được của câu hỏi

Toàn bộ pipeline suy luận được điều phối thông qua một đối tượng trạng thái chia sẻ **WorkflowState**, sử dụng khung điều phối **LangGraph** [15] để xây dựng đồ thị có hướng. Mỗi trạng thái s gồm hai nhóm trường: trường đầu vào $\mathcal{I}$ được khởi tạo từ dữ liệu bài đọc, và trường trung gian $\mathcal{O}$ được cập nhật bởi các tác tử trong quá trình xử lý:

$$\mathcal{I} = \{content, n, max_loops\} \quad (3.1)$$

$$\mathcal{O} = \{questions\} \quad (3.2)$$

Pipeline hoạt động theo hai vòng lặp kiểm soát chất lượng (Hình 3.1). Vòng lặp thứ nhất đảm bảo câu hỏi đúng cấp độ nhận thức: Tác tử *Sinh câu hỏi* (A_{gen}^l) tạo câu hỏi cho cấp độ l , sau đó Tác tử *Phân loại* (A_{cls}) xác minh cấp độ; câu hỏi bị từ chối sẽ được sinh lại ở vòng tiếp theo. Vòng lặp thứ hai đảm bảo câu hỏi có thể giải được: Tác tử *Học sinh* (A_{stu}) thử giải câu hỏi, nếu thất bại thì Tác tử *Hiệu chỉnh* (A_{fix}) sửa phương án rồi đưa lại cho Tác tử *Học sinh* kiểm định. Mỗi câu hỏi có số lần thử lại tối đa có thể cấu hình; nếu vượt quá giới hạn, câu hỏi sẽ được đánh dấu lỗi và loại khỏi kết quả cuối cùng.

3.2 Luồng xử lý đa tác tử

3.2.1 Tác tử *Sinh câu hỏi*

Hệ thống triển khai ba tác tử *sinh câu hỏi* hoạt động song song, mỗi tác tử chịu trách nhiệm cho một cấp độ nhận thức $l \in \{1, 2, 3\}$ theo thang đo Bloom. Mỗi tác tử nhận đoạn văn bản nguồn, số lượng câu hỏi cần sinh, cùng hai tập mẫu tham chiếu từ các vòng lặp trước:

$$A_{\text{gen}}^l : (content, n_l, \mathcal{P}_l, \mathcal{N}_l) \longrightarrow \{q_1, q_2, \dots, q_{n_l}\} \quad (3.3)$$

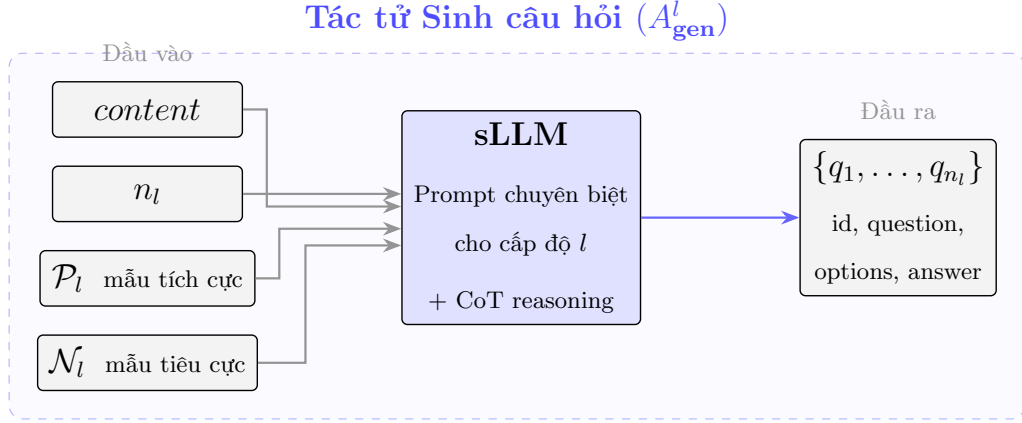
trong đó n_l là số câu hỏi cần sinh cho cấp độ l , $\mathcal{P}_l$ là tập mẫu tích cực và $\mathcal{N}_l$ là tập mẫu tiêu cực. Mỗi câu hỏi q_i được biểu diễn dưới dạng bộ:

$$q_i = (id, question, options \in \mathbb{S}^4, correct_idx \in \{0, 1, 2, 3\}, level = l, status) \quad (3.4)$$

Prompt của mỗi tác tử được thiết kế chuyên biệt cho cấp độ nhận thức tương ứng: tác tử *Cấp 1* yêu cầu câu hỏi trích xuất thông tin trực tiếp từ văn bản; tác tử *Cấp 2* đòi hỏi câu hỏi suy luận ngầm, đáp án phải được diễn đạt lại hoàn toàn so với văn bản gốc; tác tử *Cấp 3* yêu cầu câu hỏi lập luận đa bước theo chuẩn LSAT/GMAT, kết nối các thông tin rải rác trong nhiều đoạn văn.

Một kỹ thuật quan trọng trong prompt của tác tử *sinh câu hỏi* là cơ chế học từ mẫu tham chiếu. Trước khi sinh câu hỏi, mô hình nhận được danh sách các mẫu

tích cực $\mathcal{P}_l$ (câu hỏi đã đạt chuẩn từ các vòng trước) và các mẫu tiêu cực $\mathcal{N}_l$ (câu hỏi đã bị từ chối vì sai cấp độ). Prompt hướng dẫn mô hình mô phỏng phong cách và độ khó của mẫu tích cực, đồng thời tránh lặp lại các lỗi từ mẫu tiêu cực. Cơ chế này hoạt động như một dạng few-shot learning có phản hồi, trong đó chất lượng câu hỏi được cải thiện dần qua từng vòng lặp. Hình 3.2 minh họa luồng xử lý bên trong tác tử.



Hình 3.2: Luồng xử lý bên trong Tác tử Sinh câu hỏi: nhận văn bản nguồn và mẫu tham chiếu, sinh câu hỏi qua sLLM với prompt chuyên biệt theo cấp độ

3.2.2 Tác tử *Phân loại*

Tác tử *Phân loại* có nhiệm vụ xác minh rằng mỗi câu hỏi được sinh ra phù hợp với cấp độ nhận thức mong muốn:

$$A_{\text{cls}} : (\text{content}, \{q_1, \dots, q_m\}) \longrightarrow \{(id_i, \hat{l}_i)\}_{i=1}^m \quad (3.5)$$

trong đó $\hat{l}_i$ là cấp độ nhận thức do tác tử dự đoán cho câu hỏi q_i .

Để triệt tiêu các rủi ro thiên lệch trong Tác tử *Phân loại*, kỹ thuật xáo trộn ngẫu nhiên và gán mã định danh tạm thời được tích hợp như một cơ chế phòng vệ thiết yếu. Các bằng chứng thực nghiệm đã chứng minh rằng các mô hình ngôn ngữ, kể cả ở quy mô lớn, đều mang một khiếm khuyết hệ thống gọi là thiên kiến vị trí (position bias) khi hoạt động trong vai trò giám khảo[32]; chúng dễ dàng bị đánh lừa và ưu ái thông tin thuần túy dựa trên thứ tự vật lý của nó trong lời nhắc. Bằng cách gỡ bỏ mã định danh toàn cục và xáo trộn vị trí các câu hỏi ở mỗi vòng lặp trước khi đưa vào mô hình, kiến trúc này cắt đứt mọi mối tương quan giả. Điều

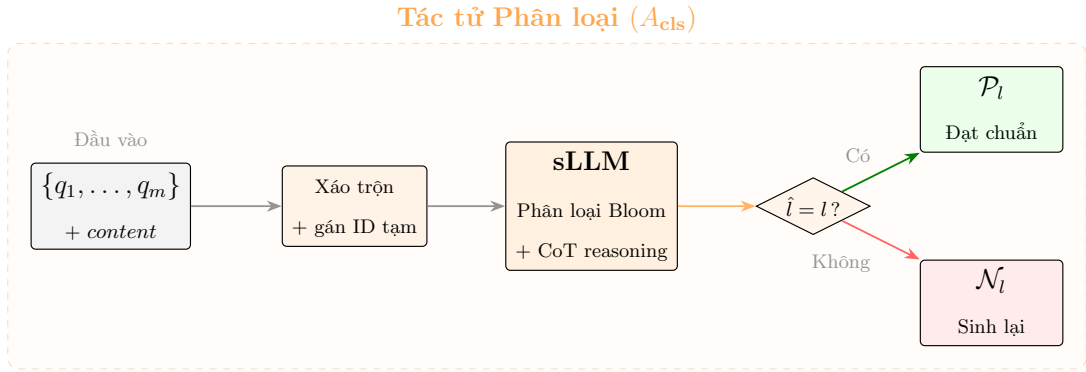
này buộc mô hình không thể dựa vào vị trí hay số thứ tự để gian lận, mà phải thực sự phân tích các đặc trưng ngữ nghĩa sâu của văn bản để đánh giá cấp độ nhận thức.

Kết quả phân loại trực tiếp cập nhật hai tập mẫu tham chiếu cho tác tử *sinh câu hỏi* ở vòng lặp tiếp theo. Với mỗi câu hỏi q ở cấp độ mục tiêu l :

$$\mathcal{P}_l^{(t+1)} = \mathcal{P}_l^{(t)} \cup \{q.question \mid \hat{l}(q) = l\} \quad (3.6)$$

$$\mathcal{N}_l^{(t+1)} = \mathcal{N}_l^{(t)} \cup \{q.question \mid \hat{l}(q) \neq l\} \quad (3.7)$$

trong đó $\mathcal{P}_l^{(t)}$ và $\mathcal{N}_l^{(t)}$ lần lượt là tập mẫu tích cực và tiêu cực tại vòng lặp t . Câu hỏi đạt chuẩn ($\hat{l} = l$) được chuyển sang Tác tử *Học sinh*; câu hỏi bị từ chối ($\hat{l} \neq l$) được đánh dấu thất bại và nút điều phối sẽ yêu cầu tác tử *sinh câu hỏi* tạo lại số lượng thiếu hụt. Hình 3.3 minh họa luồng xử lý bên trong tác tử.



Hình 3.3: Luồng xử lý bên trong Tác tử Phân loại: xáo trộn câu hỏi chống thiên lệch, phân loại cấp độ Bloom, và tách kết quả thành tập đạt chuẩn và tập sinh lại

3.2.3 Tác tử *Học sinh*

Tác tử *Học sinh* mô phỏng quá trình một học sinh giải từng câu hỏi, giúp phát hiện các câu hỏi mơ hồ, có nhiều đáp án đúng, hoặc có cấu trúc kém:

$$A_{stu} : (content, \{q_1, \dots, q_m\}) \longrightarrow \{(id_i, C_i, reason_i)\}_{i=1}^m \quad (3.8)$$

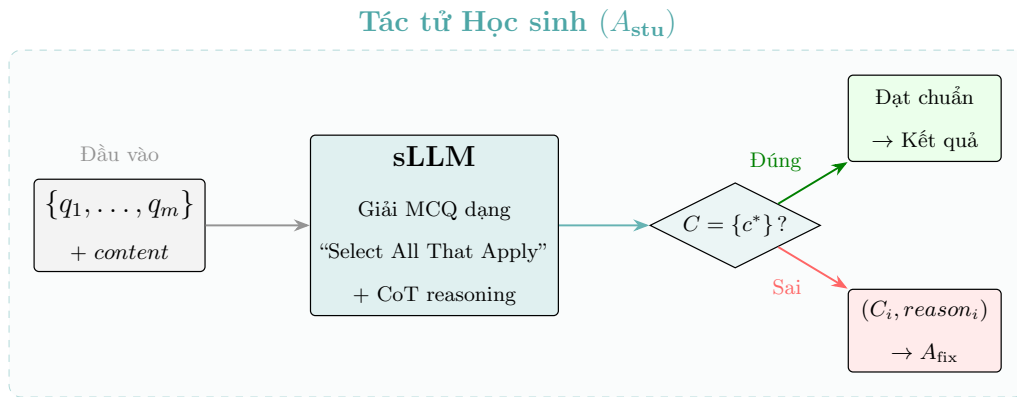
trong đó $C_i \subseteq \{A, B, C, D\} \cup \{\text{NONE}\}$ là tập hợp các phương án được tác tử lựa chọn cho câu hỏi q_i .

Một quyết định thiết kế cốt lõi trong Tác tử *Học sinh* là việc sử dụng định dạng trắc nghiệm nhiều lựa chọn đúng (Select All That Apply - SATA). Các nghiên cứu đánh giá hiện đại chỉ ra rằng định dạng trắc nghiệm một lựa chọn truyền thống thường kích hoạt độ chệch lựa chọn (selection bias), tạo điều kiện cho LLM “học phím tắt” và dùng phương pháp loại trừ để chọn phương án “trông hợp lý nhất”. Bằng cách chuyển sang định dạng SATA, mô hình bị ép buộc phải thẩm định tính chân lý của từng phương án một cách độc lập tuyệt đối so với văn bản nguồn. Cơ chế khắt khe này vô hiệu hóa các suy luận đoán mò, biến Tác tử *Học sinh* thành một màng lọc chẩn đoán độ mơ hồ cực kỳ nhạy bén, giúp hệ thống phát hiện và loại bỏ các câu hỏi vi phạm tính đơn nghĩa (nhiều đáp án đúng) hoặc có các phương án nhiễu quá dễ dãi[35].

Câu hỏi được xác nhận đạt chuẩn khi và chỉ khi tác tử lựa chọn đúng một phương án duy nhất trùng với đáp án chính xác:

$$passed(q) = \begin{cases} true & \text{nếu } C(q) = \{c^*\} \\ false & \text{ngược lại} \end{cases} \quad (3.9)$$

trong đó c^* là ký hiệu phương án đúng của câu hỏi q . Nếu tác tử trả lời sai, câu hỏi được chuyển tới Tác tử *Hiệu chỉnh* cùng với lý do giải thích. Hình 3.4 minh họa luồng xử lý bên trong tác tử.



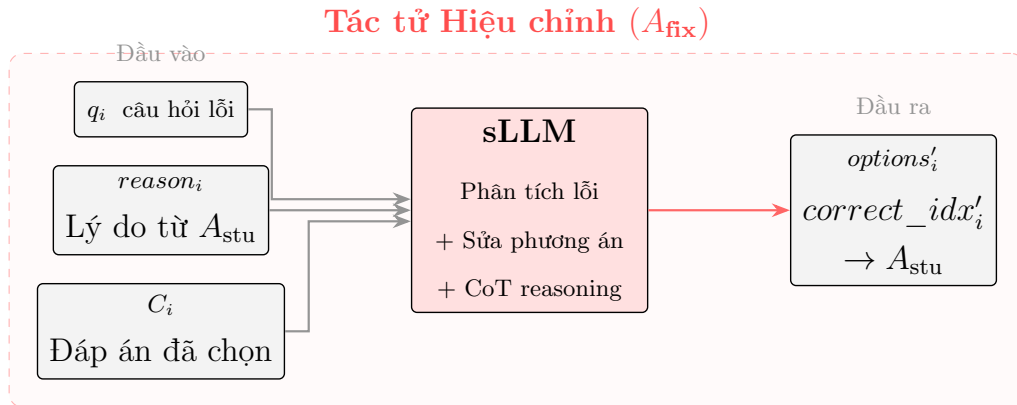
Hình 3.4: Luồng xử lý bên trong Tác tử Học sinh: giải MCQ dạng nhiều lựa chọn đúng, phân luồng câu hỏi đạt chuẩn và câu hỏi cần hiệu chỉnh kèm lý do

3.2.4 Tác tử *Hiệu chỉnh*

Tác tử *Hiệu chỉnh* tiếp nhận các câu hỏi mà Tác tử *Học sinh* đã trả lời sai, cùng với lý do giải thích:

$$A_{\text{fix}} : (content, \{(q_i, C_i, reason_i)\}) \longrightarrow \{(id_i, options'_i, correct_idx'_i)\} \quad (3.10)$$

Ràng buộc thiết kế cốt lõi của tác tử này là chỉ được phép sửa đổi các phương án trả lời và đáp án đúng, trong khi bắt buộc giữ nguyên thân câu hỏi và cấp độ nhận thức. Ràng buộc này ngăn chặn hệ thống tạo ra câu hỏi hoàn toàn mới thay vì sửa chữa câu hỏi hiện có. Tác tử sử dụng lý do giải thích từ Tác tử *Học sinh* để xác định nguyên nhân lỗi, chẳng hạn phương án nhiễu quá hiển nhiên, nhiều phương án cùng đúng, hoặc đáp án chính thức không chính xác, và thực hiện sửa chữa có mục tiêu. Sau khi sửa chữa, câu hỏi được đưa trở lại Tác tử *Học sinh* để tái kiểm định. Hình 3.5 minh họa luồng xử lý bên trong tác tử.



Hình 3.5: Luồng xử lý bên trong Tác tử Hiệu chỉnh: nhận câu hỏi lỗi kèm lý do từ Tác tử Học sinh, chỉ sửa phương án và đáp án, đưa lại để tái kiểm định

3.3 Một số kỹ thuật mở rộng

3.3.1 Suy luận chuỗi tư duy

Toàn bộ các tác tử trong pipeline đều áp dụng kỹ thuật suy luận chuỗi tư duy (CoT - Chain-of-Thought [34]) trước khi đưa ra kết quả. Cụ thể, mỗi prompt đều yêu cầu mô hình trình bày chuỗi lập luận thông qua trường **Reason** trong định dạng đầu ra bắt buộc, trước khi đưa ra câu trả lời hoặc quyết định cuối cùng.

Thiết kế này được thể hiện nhất quán trong prompt của từng tác tử:

- **Tác tử *Sinh câu hỏi*:** Trường Reason yêu cầu mô hình giải thích chiến lược tạo câu hỏi và các phương án trước khi sinh nội dung.
- **Tác tử *Phân loại*:** Trường Reason yêu cầu mô hình phân tích cấp độ nhận thức dựa trên bằng chứng cụ thể trước khi đưa ra phán quyết.
- **Tác tử *Học sinh*:** Trường Reason yêu cầu mô hình trình bày chứng minh từng bước trong ba câu, giải thích lý do chọn hoặc loại từng phương án.
- **Tác tử *Hiệu chỉnh*:** Trường Reason yêu cầu mô hình phân tích nguyên nhân lỗi của phương án gốc trước khi đề xuất bản sửa.

Kỹ thuật suy luận chuỗi tư duy đặc biệt quan trọng khi sử dụng các sLLM, vì các mô hình có quy mô nhỏ thường gặp khó khăn trong các tác vụ đòi hỏi suy luận nhiều bước. Bằng cách buộc mô hình trình bày rõ quá trình tư duy trước khi đưa ra kết quả, kỹ thuật này giúp giảm lỗi logic, tăng tính nhất quán của đầu ra, đồng thời cho phép truy vết và gỡ lỗi khi cần thiết.

3.3.2 Định dạng đầu ra của tác tử

Toàn bộ pipeline sử dụng đầu ra dạng văn bản thuần có cấu trúc thay vì yêu cầu mô hình trả về định dạng JSON. Cụ thể, mỗi tác tử trả về kết quả theo cấu trúc khóa-giá trị cố định như được minh họa trong Hình 3.6:

ID: [ID]
Reason: [...]
Question: [text]
A: [option A]
B: [option B]
C: [option C]
D: [option D]
Answer: [A|B|C|D]

Hình 3.6: Cấu trúc định dạng đầu ra chuẩn của tác tử *Sinh câu hỏi*

Quyết định thiết kế sử dụng văn bản thuần có cấu trúc thay vì JSON xuất phát từ giới hạn thực nghiệm của các mô hình ngôn ngữ nhỏ (sLLMs) quy mô dưới 15 tỷ tham số. Việc sinh cấu trúc JSON lồng ghép yêu cầu mô hình phân bổ một lượng lớn năng lực tính toán để duy trì tính hợp lệ của cú pháp (dấu ngoặc, ký tự thoát), dẫn đến sự nhiễu loạn và rủi ro sụp đổ độ tin cậy đầu ra rất cao khi bộ phân tích cú pháp hạ nguồn thất bại[8]. Ngược lại, định dạng văn bản thuần tuần tự (khóa-giá trị) cộng hưởng tự nhiên với cơ chế sinh từ từ hồi quy của mô hình. Thiết kế này giải phóng tài nguyên nhận thức của sLLM để tập trung vào suy luận logic, đồng thời cung cấp khả năng chịu lỗi vượt trội cho đường ống trích xuất dữ liệu, ngay cả khi mô hình vô tình sinh thêm các ký tự định dạng dư thừa[8].

Chương 4

Phương pháp đánh giá

Chương này trình bày chi tiết về thiết kế thực nghiệm nhằm kiểm chứng hiệu quả của các phương pháp đã đề xuất. Nội dung bao gồm việc giới thiệu bộ dữ liệu được sử dụng, quy trình tổng hợp dữ liệu ảnh nhiều cho bài toán OCR, các thiết lập kỹ thuật cho mô hình, cũng như các tiêu chí và phương pháp đánh giá tự động (LLM-as-a-Judge) được áp dụng.

4.1 Bộ dữ liệu

RACE (Reading Comprehension Dataset From Examinations) [14] là một bộ dữ liệu quy mô lớn trong lĩnh vực xử lý ngôn ngữ tự nhiên. Khác với bộ dữ liệu **SQuAD** [26] (chủ yếu dựa trên Wikipedia), RACE được xây dựng trực tiếp từ đề thi của các kỳ thi tiếng Anh thực tế dành cho học sinh trung học cơ sở và trung học phổ thông tại hệ thống giáo dục Trung Quốc.

Hai lý do chính khiến nghiên cứu sử dụng RACE làm bộ dữ liệu đánh giá mô hình là: Độ phức tạp và đa dạng ngữ nghĩa: Văn bản trong RACE thường bao trùm nhiều lĩnh vực trong đời sống: giáo dục, tin tức, văn hóa, chính trị, khoa học và các câu chuyện luân lý. Chính vì thế, đặc điểm dữ liệu và văn phong của văn bản trong RACE vô cùng sát với các tài liệu trong thực tế. Các văn bản trong RACE có độ dài tiêu chuẩn từ 300 đến 500 từ, độ dài này đủ lớn để chứa đựng lượng thông tin cho một bài kiểm tra: các chuỗi sự kiện phức tạp, luận điểm, nhưng vẫn đảm bảo không làm quá tải cửa sổ ngữ cảnh (context window) của các sLLM.

Chiều sâu logic và tính đa nghĩa của văn bản gốc: Không giống các văn bản bách khoa toàn thư chỉ liệt kê dữ liệu bề mặt, các văn bản của RACE được các chuyên gia giáo dục tuyển chọn chuyên biệt nhằm mục đích đánh giá nhận thức của người học. Cấu trúc của văn bản ẩn chứa nhiều tầng nghĩa, tích hợp các chuỗi nhân quả, thái độ ngầm định của tác giả và các mệnh đề gây nhiễu. Nếu nghiên cứu lựa chọn một bộ dữ liệu văn bản thiếu chiều sâu, các nỗ lực đánh giá LLM sinh câu hỏi tự duy cao đều sẽ thất bại.

Bộ dữ liệu RACE bao gồm khoảng 28.000 đoạn văn và gần 100.000 câu hỏi trắc nghiệm, được chia thành hai tập con theo cấp độ: RACE-Middle (dành cho học sinh trung học cơ sở) và RACE-High (dành cho học sinh trung học phổ thông). Tập RACE-High chứa các đoạn văn có độ phức tạp cao hơn, đòi hỏi khả năng suy luận sâu hơn, do đó phù hợp hơn với mục tiêu đánh giá khả năng sinh câu hỏi đa cấp độ nhận thức của nghiên cứu này.

Trong phạm vi thực nghiệm, nghiên cứu lấy mẫu ngẫu nhiên 100 đoạn văn từ tập RACE-High làm dữ liệu đầu vào cho toàn bộ các thí nghiệm sinh câu hỏi. Bảng 4.1 trình bày thống kê tổng quan về bộ dữ liệu RACE và mẫu được sử dụng.

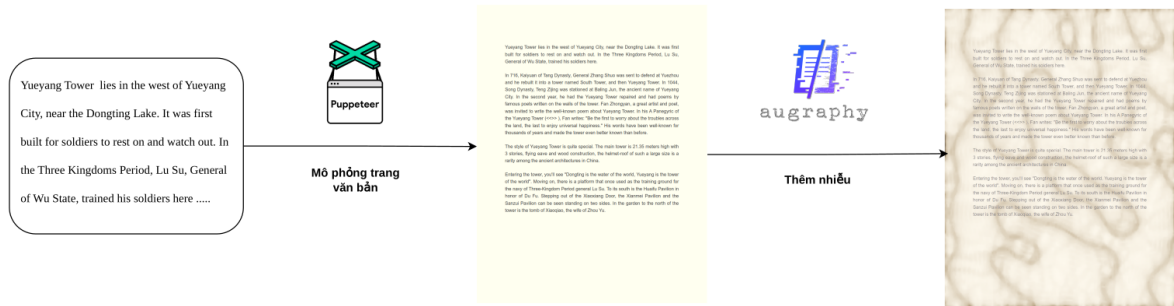
Đặc điểm	RACE-Middle	RACE-High	Mẫu thực nghiệm
Số đoạn văn	~6.400	~21.600	100
Số câu hỏi	~25.400	~72.000	-
Nguồn gốc	Đề thi THCS	Đề thi THPT	Lấy mẫu từ RACE-High
Độ dài trung bình (từ)	250-350	300-500	300-500

Bảng 4.1: Thống kê bộ dữ liệu RACE và mẫu thực nghiệm

4.2 Quy trình tổng hợp bộ dữ liệu OCR

Để đánh giá chính xác năng lực nhận dạng văn bản (OCR) của các mô hình thị giác - ngôn ngữ (VLM) trong điều kiện thực tế - nơi tài liệu thường ở dạng ảnh chụp, sách giáo khoa in, hoặc ghi chép tay với chất lượng quang học đa dạng - nghiên cứu tiến hành xây dựng một bộ dữ liệu ảnh giả lập chuyên biệt.

Thay vì thu thập và gán nhãn ảnh chụp thủ công (phương pháp tốn kém tài nguyên và khó kiểm soát độc lập các biến số nhiễu), hệ thống triển khai một quy trình sinh dữ liệu tổng hợp (synthetic data generation). Quá trình này đảo ngược trình tự của bài toán OCR truyền thống trong khâu chuẩn bị dữ liệu: bắt đầu từ các đoạn văn bản chuẩn (ground truth) trích xuất từ bộ dữ liệu gốc, hệ thống chủ động kết xuất (render) thành hình ảnh, sau đó áp dụng các phép biến đổi có kiểm soát để mô phỏng tác nhân gây nhiễu thực tế (Hình 4.1). Cách tiếp cận này giúp tạo ra nguồn dữ liệu đánh giá dồi dào, đa dạng và có sẵn nhãn chính xác tuyệt đối để đo lường hiệu năng của các VLM.



Hình 4.1: Sơ đồ quy trình tăng cường dữ liệu ảnh nhiễu: từ văn bản gốc qua kết xuất ảnh sạch, áp dụng nhiễu vật lý và quang học, đến ảnh giả lập tài liệu thực tế

4.2.1 Sinh ảnh sạch từ văn bản

Giai đoạn đầu tiên là khởi tạo các hình ảnh kỹ thuật số mang độ sắc nét tuyệt đối từ văn bản gốc. Nhằm đảm bảo tính đa dạng về mặt thị giác và cấu trúc tài liệu, nghiên cứu không sử dụng các thư viện đồ họa 2D cơ bản (như PIL), mà tích hợp công cụ trình duyệt không giao diện (headless browser) thông qua thư viện Pyppeteer [20]. Phương pháp này tận dụng sức mạnh của CSS và DOM để kiểm soát chính xác và linh hoạt bố cục hình ảnh. Quá trình này gồm ba bước cốt lõi:

1. **Khởi tạo không gian và định dạng kiểu chữ:** Hệ thống chọn ngẫu nhiên các kích thước khung hình tiêu chuẩn (A4, A5, US Letter, FHD, HD, v.v.). Phông chữ được lấy ngẫu nhiên từ kho dữ liệu Google Fonts [9], chia làm hai nhóm đặc thù: nhóm in chuẩn (Roboto, Merriweather) để mô phỏng tài liệu in ấn, và nhóm viết tay (Dancing Script, Caveat) để giả lập các bản ghi chép cá nhân. Các tham số về độ đậm (weight), màu chữ và màu nền cũng được cấu hình ngẫu nhiên.
2. **Tự động hóa mật độ hiển thị:** Dựa trên tỷ lệ diện tích khung hình và số lượng ký tự, thuật toán linh hoạt chia văn bản thành nhiều cột (từ 1 đến 3 cột), đồng thời tinh chỉnh khoảng cách lề (padding), độ giãn dòng (line height) và khoảng cách đoạn (paragraph spacing). Cơ chế này giúp sinh ra các tài liệu có cấu trúc từ đơn giản đến phức tạp.
3. **Khớp khung hình động:** Để khắc phục tình trạng văn bản bị cắt xén hoặc tràn viền khi chụp ảnh toàn trang, một mã JavaScript được nhúng trực tiếp vào tài liệu HTML/CSS. Thuật toán này tự động co giãn cỡ chữ (font-size)

sao cho toàn bộ nội dung hiển thị vừa vặn trong một khung hình duy nhất (không làm xuất hiện thanh cuộn), trước khi công cụ Pyppeteer tiến hành chụp và lưu trữ ảnh.

4.2.2 Tạo dữ liệu ảnh nhiều mô phỏng thực tế

Trong thực tiễn triển khai, hình ảnh đầu vào từ người dùng hiếm khi đạt chất lượng quang học lý tưởng do hạn chế của thiết bị hoặc điều kiện môi trường. Để thu hẹp khoảng cách giữa dữ liệu phòng thí nghiệm và môi trường thực tế, hình ảnh sạch từ bước trước được đưa qua quy trình giả lập nhiễu.

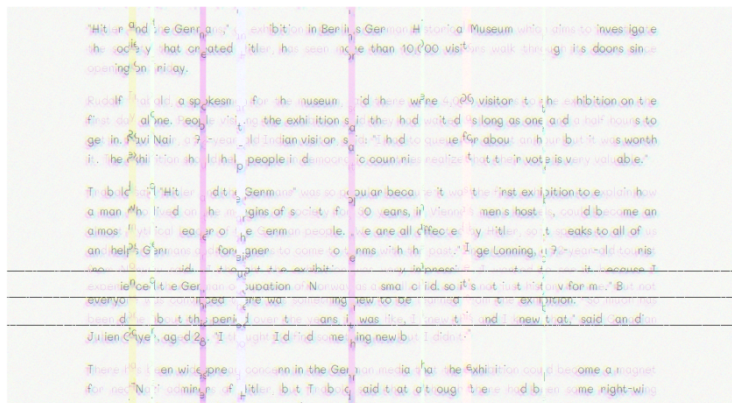
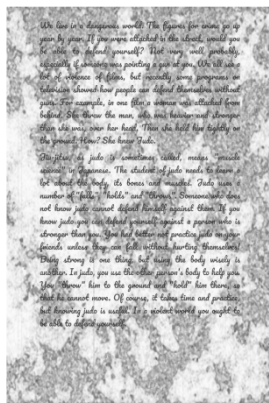
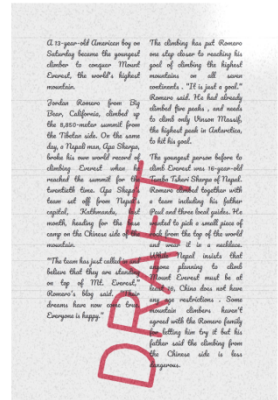
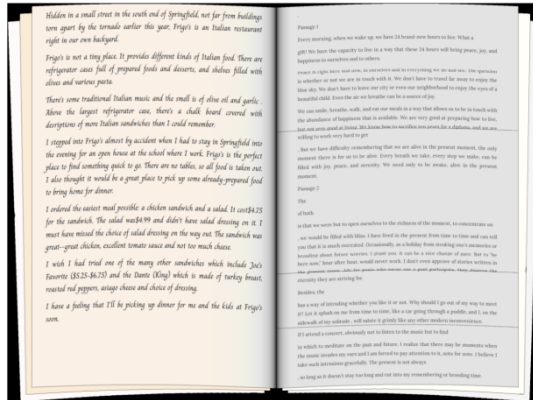
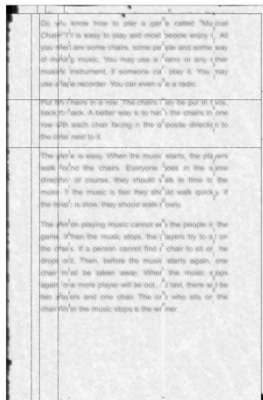
Nghiên cứu ứng dụng **Augraphy** [12] - một thư viện chuyên dụng cho bài toán tăng cường dữ liệu tài liệu vật lý - để thiết lập các lớp nhiễu phức tạp:

- **Nhiều vật liệu và thiết bị in ấn:** Hệ thống chèn thêm các dải nhiễu cấu trúc như vết nhòe mực, đứt nét hoặc mờ chữ do tình trạng cặn mực máy in. Nền ảnh gốc được phủ thêm các họa tiết bề mặt giấy, vết ố vàng theo thời gian hoặc văn bản in chìm ở mặt sau (bleed-through). Những yếu tố này đòi hỏi mô hình VLM phải có khả năng phân tách chính xác giữa tín hiệu văn bản và tín hiệu nhiễu nền.
- **Nhiều quang học và tác động cơ học:** Hình ảnh chịu các phép biến đổi về độ phơi sáng, bóng đổ vật lý, nhiễu hạt ISO (khi chụp thiếu sáng) và hiện tượng lóa (glare). Ngoài ra, hệ thống mô phỏng các lỗi nén ảnh, biến dạng hình học (giấy nhăn, cong vênh ở gáy sách), cũng như các can thiệp thủ công từ học sinh (nét gạch chân, vết bút dạ quang, hoặc nét vẽ nguệch ngoạc che khuất một phần văn bản).

Qua quy trình này, mỗi đoạn văn bản gốc được biến đổi thành một phiên bản tài liệu vật lý đặc thù (Hình 4.2), tạo ra một môi trường kiểm thử khắc nghiệt để bộc lộ rõ năng lực thực sự của các mô hình OCR.

4.2.3 Kiểm thử nhận dạng văn bản

Sau khi bộ dữ liệu hoàn thiện, các mô hình OCR (cả VLM chuyên dụng cỡ nhỏ lẫn LLM đa phương thức thương mại) được gọi thông qua các cổng giao tiếp trung



Hình 4.2: Các mẫu ảnh tài liệu tổng hợp minh họa các loại nhiễu được áp dụng: nhiễu vật liệu (vết ó vàng, nhòe mực), nhiễu quang học (bóng đổ, nhiễu hạt), và biến dạng hình học (cong vênh trang)

gian như OpenRouter [25] và Novita AI [23]. Giải pháp này giúp hệ thống linh hoạt thay đổi và đối chiếu chéo hiệu năng giữa các kiến trúc mô hình tiên tiến nhất mà không phụ thuộc vào hạ tầng phần cứng cục bộ.

4.3 Thiết lập thực nghiệm

4.3.1 Các mô hình sử dụng

Nghiên cứu đánh giá tổng cộng 12 mô hình chia thành ba nhóm: nhóm mô hình ngôn ngữ lớn mã nguồn mở quy mô nhỏ (sLLM) dùng cho sinh câu hỏi, nhóm VLM quy mô nhỏ dùng chuyên biệt cho OCR, và nhóm mô hình ngôn ngữ lớn thương mại (LLM) dùng cho cả hai nhiệm vụ.

Bảng 4.2 tóm tắt các mô hình sinh câu hỏi được sử dụng trong thực nghiệm. Các mô hình sLLM có kích thước tham số từ 8 đến 14 tỷ, phù hợp với mục tiêu triển khai trên phần cứng phổ thông. Các mô hình thương mại được đưa vào làm mô hình cơ sở để đánh giá mức độ cạnh tranh của giải pháp MAS kết hợp sLLM.

Nhóm	Mô hình	Số tham số
sLLM mã nguồn mở	Gemma 3 12B IT [30]	12B
	Qwen3 8B [36]	8B
	Phi-4 [1]	14B
	Nemotron Nano 9B v2 [24]	9B
	Llama 3.1 8B Instruct [11]	8B
LLM thương mại	Gemini 3 Flash [29]	-
	GPT-5 Mini [27]	-
	Claude Haiku 4.5 [3]	-

Bảng 4.2: Các mô hình sinh câu hỏi sử dụng trong thực nghiệm

Bảng 4.3 liệt kê các mô hình được sử dụng cho nhiệm vụ OCR, bao gồm bốn VLM nhỏ gọn chuyên dụng và ba mô hình thương mại đa phương thức. Các mô hình thương mại đã được mô tả tại Bảng 4.2; trong phần này chúng được đánh giá thêm về năng lực OCR.

Nhóm	Mô hình	Mô tả
VLM nhỏ gọn	DeepSeek OCR 2 [33]	Mô hình tối ưu cho OCR tài liệu
	DotOCR 1.5 [16]	Mô hình nhẹ chuyên dụng cho văn bản in
	MinerU 2.5 [22]	Mô hình trích xuất văn bản từ tài liệu đa dạng
	PaddleOCR VL 1.5 [6]	Mô hình dựa trên nền tảng PaddlePaddle
LLM thương mại	Gemini 3 Flash [29]	Mô hình đa phương thức của Google
	GPT-5 Mini [27]	Mô hình đa phương thức của OpenAI
	Claude Haiku 4.5 [3]	Mô hình đa phương thức của Anthropic

Bảng 4.3: Các mô hình OCR sử dụng trong thực nghiệm

4.3.2 Cấu hình thực nghiệm

Toàn bộ thực nghiệm được triển khai trên nền tảng Python 3.12, sử dụng thư viện **LangGraph** [15] để điều phối luồng hoạt động của hệ thống đa tác tử. Các mô hình được gọi thông qua OpenRouter API, cho phép truy cập thống nhất đến cả mô hình mã nguồn mở và thương mại.

Các siêu tham số chính được thiết lập như sau:

- Nhiệt độ (temperature): 0,0 cho các tác tử chính (tác tử *Sinh câu hỏi*, tác tử *Phân loại*, tác tử *Hiệu chỉnh*) nhằm đảm bảo tính nhất quán và tái lập được của kết quả; 0,3 cho tác tử *Học sinh* nhằm tạo sự đa dạng trong cách trả lời.
- Số token tối đa (max_tokens): 8.192 token cho mỗi lần gọi mô hình.
- Số vòng lặp tối đa (max_loops): Giới hạn ở 8 vòng cho mỗi chu trình phản hồi giữa các tác tử.

Về quy mô thực nghiệm, mỗi đoạn văn đầu vào được yêu cầu sinh $N = 5$ câu hỏi cho mỗi cấp độ nhận thức (Nhớ, Hiểu, Phân tích), tổng cộng 15 câu hỏi trên mỗi đoạn văn. Với 100 đoạn văn mẫu, mỗi cấu hình mô hình tạo ra tối đa 1.500 câu hỏi. Chi phí sử dụng API được theo dõi tự động thông qua trường metadata của phản hồi từ OpenRouter API.

Đối với quy trình đánh giá chất lượng bằng phương pháp LLM giám khảo (đã trình bày tại Chương 2), nghiên cứu sử dụng Gemini 3 Flash làm mô hình giám khảo

cho toàn bộ các thí nghiệm. Lựa chọn này dựa trên ba tiêu chí: (i) hiệu suất suy luận cao và ổn định trên các tác vụ đánh giá ngôn ngữ, (ii) cửa sổ ngữ cảnh đủ lớn để xử lý đồng thời đoạn văn nguồn và toàn bộ câu hỏi, và (iii) chi phí hợp lý cho việc đánh giá quy mô lớn (1.500 câu hỏi mỗi cấu hình).

4.4 Phương pháp và Tiêu chí Đánh giá

Bất kỳ hệ thống sinh tự động nào cũng yêu cầu một cơ chế đánh giá độ tin cậy và mức độ hiệu quả của dữ liệu đầu ra. Trong nghiên cứu này, hệ thống trải qua hai khâu đánh giá độc lập tương ứng với hai luồng tác vụ chính: đánh giá chất lượng câu hỏi trắc nghiệm được sinh ra và đánh giá độ chính xác của quy trình nhận diện văn bản.

4.4.1 Đánh giá chất lượng câu hỏi trắc nghiệm

Đối với bài toán sinh câu hỏi trắc nghiệm tự động, các thang đo truyền thống như BLEU hay ROUGE tỏ ra kém hiệu quả và thiếu tin cậy vì chỉ so sánh sự trùng lặp từ vựng bề mặt mà bỏ qua tính sư phạm và ý nghĩa thực tế của câu hỏi. Để giải quyết vấn đề này, nghiên cứu phát triển một khung đánh giá tự động dựa trên ba tiêu chí sư phạm độc lập, kết hợp với phương pháp LLM-as-a-Judge.

Các tiêu chí sư phạm

Mỗi tiêu chí dưới đây đo lường một khía cạnh sư phạm của câu hỏi, đảm bảo đầu ra thỏa mãn các yêu cầu khắt khe của giáo dục:

1. **Khả năng giải quyết dựa trên ngữ cảnh (Solvability):** Tiêu chí này kiểm định tính đóng của không gian thông tin. Theo lý thuyết, một câu hỏi đạt chuẩn bắt buộc phải chứa duy nhất một đáp án đúng, và đáp án này phải được suy luận hoàn toàn ra từ các thông tin nằm bên trong đoạn văn bản được cung cấp. Yêu cầu này nghiêm cấm hiện tượng ảo giác của LLM khi sinh ra các câu trả lời dựa trên kiến thức bên ngoài. Nếu người đọc không thể trả lời câu hỏi ngay trên văn bản đầu vào, câu hỏi đó bị đánh giá là vi phạm.
2. **Độ bám sát phân loại nhận thức (Alignment):** Tiêu chí này đo lường

xem mức độ nhận thức (độ khó) của câu hỏi sinh ra có đúng với cấu hình yêu cầu từ trước hay không. Hệ thống sẽ đối chiếu câu hỏi sinh ra với thang đo Bloom. Ví dụ, nếu LLM được giao nhiệm vụ sinh ra một câu hỏi Cấp 3 mà lại đưa ra một câu hỏi Cấp 1, đó là vi phạm do lệch pha về độ sâu nhận thức.

3. **Chất lượng phương án nhiễu (Distractor Quality)**: Tiêu chí này kiểm định độ sắc bén của câu hỏi. Một phương án sai hợp lệ phải đồng thời thỏa mãn hai điều kiện: trái ngược hoàn toàn so với đáp án đúng về mặt bản chất (không tạo ra tranh cãi hoặc tính đa nghĩa); và có khả năng đánh lừa những người học không nắm vững kiến thức. Các lựa chọn nhiễu hiển nhiên sai hoặc lệch ngữ cảnh sẽ bị loại bỏ.

Phương pháp LLM-as-a-Judge

Khác với cách tiếp cận truyền thống của G-Eval (nơi LLM giám khảo chấm điểm theo bộ tiêu chí tĩnh), nghiên cứu này triển khai phương pháp đánh giá dựa trên hành vi thực tế. LLM giám khảo được đặt vào vai trò một học sinh chuyên nghiệp để trực tiếp giải bài và phân loại câu hỏi, từ đó đo lường chất lượng thông qua kết quả hành động. Quy trình này gồm hai giai đoạn:

Giai đoạn 1 - Đánh giá Khả năng giải quyết và Độ bám sát:

LLM giám khảo nhận đầu vào gồm đoạn văn bản nguồn và bộ câu hỏi cần đánh giá (không kèm đáp án đúng). Mô hình phải tự giải từng câu hỏi. Bài kiểm tra được thiết kế dưới dạng nhiều lựa chọn đúng (mỗi câu hỏi có thể có một, nhiều, hoặc không có phương án đúng) nhằm buộc LLM đánh giá từng phương án độc lập. Với mỗi câu hỏi, LLM giám khảo đưa ra chuỗi lập luận, sau đó trả về: (1) các phương án cho là đúng, và (2) cấp độ nhận thức dự đoán.

Từ đó, hai chỉ số được tính toán:

- **Khả năng giải quyết**: Câu hỏi đạt tiêu chí khi tập hợp phương án LLM chọn trùng khớp hoàn toàn với đáp án đúng.

$$\text{Solvability}(q) = \begin{cases} 1, & \text{nếu } C_{\text{judge}}(q) = \{c^*\} \\ 0, & \text{ngược lại} \end{cases}$$

trong đó $C_{\text{judge}}(q)$ là tập hợp các phương án được LLM giám khảo chọn cho câu

hỏi q , và c^* là đáp án đúng duy nhất. Nếu chọn sai hoặc chọn nhiều phương án, câu hỏi bị đánh giá 0 (có thể do mơ hồ hoặc chứa nhiều đáp án đúng).

- **Độ bám sát phân loại nhận thức:** Đo lường mức độ phù hợp giữa cấp độ nhận thức LLM dự đoán và cấp độ yêu cầu:

$$\text{Alignment}(q) = \begin{cases} 1, & \text{nếu } l_{\text{judge}}(q) = l_{\text{target}}(q) \\ 0, & \text{ngược lại} \end{cases}$$

trong đó $l_{\text{judge}}(q)$ là cấp độ do LLM dự đoán và $l_{\text{target}}(q)$ là cấp độ yêu cầu.

Tỷ lệ chấp nhận tổng hợp (Acceptance Rate) là tỷ lệ câu hỏi đồng thời đạt cả hai tiêu chí:

$$\text{Acceptance}(q) = \text{Solvability}(q) \times \text{Alignment}(q)$$

$$\text{Acceptance Rate} = \frac{1}{N} \sum_{i=1}^N \text{Acceptance}(q_i)$$

trong đó N là tổng số câu hỏi được đánh giá. Chỉ những câu hỏi có Acceptance bằng 1 mới được đưa vào Giai đoạn 2.

Giai đoạn 2 - Đánh giá Chất lượng phương án nhiễu:

LLM giám khảo nhận đầy đủ câu hỏi, bốn phương án, đáp án đúng và cấp độ nhận thức. Nhiệm vụ là đánh giá ba phương án sai theo tiêu chuẩn từng cấp độ:

- **Cấp 1 (Truy xuất):** Mâu thuẫn trực tiếp với văn bản hoặc đưa thông tin bên ngoài vào.
- **Cấp 2 (Suy luận):** Chứa từ khóa từ văn bản nhưng mệnh đề sai, hoặc trình bày sự thật thiếu hoàn chỉnh.
- **Cấp 3 (Lập luận phản biện):** Chứa bẫy logic tinh vi (áp dụng sai quy luật, đảo ngược nhân quả, giả định không được hỗ trợ).

Gọi $v(q)$ là số phương án nhiễu hợp lệ của câu hỏi q (với $v(q) \in \{0, 1, 2, 3\}$). Chỉ số chất lượng cho mỗi cấp độ l được chuẩn hóa về khoảng $[0, 1]$:

$$\text{DQ}_l = \frac{1}{N_l} \sum_{q \in \mathcal{A}_l} \frac{v(q)}{3}$$

trong đó $\mathcal{A}_l$ là tập hợp các câu hỏi đã được chấp nhận ở cấp độ l , và $N_l = |\mathcal{A}_l|$.

Chỉ số Distractor Quality tổng thể là trung bình cộng trên ba cấp độ:

$$\text{DQ}_{\text{overall}} = \frac{1}{3} \sum_{l=1}^3 \text{DQ}_l$$

4.4.2 Đánh giá hiệu năng nhận dạng văn bản

Để đánh giá chất lượng của quy trình trích xuất văn bản từ hình ảnh giả lập nhiều, nghiên cứu sử dụng hai thang đo chuẩn công nghiệp: tỷ lệ lỗi ký tự (CER - Character Error Rate) và tỷ lệ lỗi từ (WER - Word Error Rate). Cả hai chỉ số đều dựa trên khoảng cách chỉnh sửa Levenshtein, đo lường số phép chèn (I), xóa (D) và thay thế (S) tối thiểu cần thiết để biến đổi chuỗi dự đoán thành chuỗi tham chiếu gốc:

$$\text{CER} = \frac{S_c + D_c + I_c}{N_c}$$

$$\text{WER} = \frac{S_w + D_w + I_w}{N_w}$$

trong đó N_c và N_w lần lượt là tổng số ký tự và tổng số từ trong chuỗi tham chiếu. Giá trị CER và WER càng thấp thể hiện chất lượng nhận dạng OCR càng tốt, với giá trị 0 tương ứng với việc nhận dạng hoàn hảo không có sai sót.

4.4.3 Các mô hình cơ sở

Nhằm kiểm chứng khách quan hiệu năng của phương pháp đề xuất, nghiên cứu thiết lập các mô hình cơ sở tương ứng với hai bài toán cốt lõi: sinh câu hỏi trắc nghiệm và nhận dạng văn bản.

Trong bài toán sinh câu hỏi trắc nghiệm, nghiên cứu sử dụng hai nhóm đối chứng để phục vụ các mục tiêu so sánh đa chiều:

- **Phương pháp sinh trực tiếp** : Sử dụng chính các mô hình sLLM (Gemma 3 12B, Qwen3 8B, Phi-4, Nemotron Nano 9B, Llama 3.1 8B) vận hành độc lập, không thông qua kiến trúc đa tác tử. Việc đối chiếu với nhóm cơ sở này nhằm kiểm định giả thuyết rằng sự phối hợp đa tác tử mang lại mức độ cải

thiện đáng kể và mang tính hệ thống so với giới hạn nội tại của các mô hình đơn lẻ.

- **Nhóm mô hình thương mại:** Sử dụng các mô hình ngôn ngữ lớn tiên tiến (Gemini 3 Flash, GPT-5 Mini, Claude Haiku 4.5) làm hệ quy chiếu. So sánh này nhằm đánh giá tính khả thi và khả năng cạnh tranh về chất lượng của giải pháp dùng sLLM kết hợp đa tác tử so với các nền tảng có chi phí vận hành đắt đỏ.

Trong bài toán trích xuất văn bản, các mô hình đối chứng được lựa chọn bao gồm:

- **Nhóm VLM chuyên dụng:** Sử dụng các mô hình thị giác - ngôn ngữ quy mô nhỏ được tinh chỉnh chuyên biệt cho tác vụ trích xuất tài liệu (DeepSeek OCR 2, DotOCR 1.5, MinerU 2.5, PaddleOCR VL 1.5). Đây là mốc cơ sở tiêu chuẩn để đánh giá năng lực trích xuất văn bản.
- **Nhóm mô hình đa phương thức lớn:** Khai thác các mô hình LLM thương mại có năng lực thị giác (Gemini 3 Flash, GPT-5 Mini, Claude Haiku 4.5) để thiết lập giới hạn trên về khả năng nhận dạng và kiểm chứng mức độ duy trì độ chính xác (kháng nhiễu) trong điều kiện dữ liệu ảnh phức tạp.

Chương 5

Kết quả

5.1 Kết quả sinh câu hỏi

Phần này trình bày kết quả đánh giá chất lượng câu hỏi trắc nghiệm được sinh bởi các mô hình theo ba nhóm: sLLM cơ sở (không có hệ thống đa tác tử), sLLM kết hợp đa tác tử, và LLM thương mại. Ba chỉ số đánh giá chính được sử dụng là **Acceptance** (tỷ lệ câu hỏi được chấp nhận), **Alignment** (mức độ phù hợp giữa cấp độ nhận thức sinh ra và cấp độ yêu cầu), và **Distractor Quality** (chất lượng các phương án nhiễu). Các chỉ số này được đo bằng phương pháp G-Eval với thang điểm từ 0 đến 1. Kết quả được phân tích theo ba cấp độ nhận thức: Cấp 1 (Nhớ), Cấp 2 (Hiểu) và Cấp 3 (Phân tích).

5.1.1 Kết quả nhóm sLLM cơ sở

Bảng 5.1 trình bày kết quả đánh giá của năm mô hình sLLM khi hoạt động độc lập, không có sự hỗ trợ của hệ thống đa tác tử.

Bảng 5.1: Kết quả đánh giá nhóm sLLM cơ sở trên bộ dữ liệu RACE-High

Mô hình	Cấp 1 - Nhớ			Cấp 2 - Hiểu			Cấp 3 - Phân tích		
	Acc.	Align.	Dist.	Acc.	Align.	Dist.	Acc.	Align.	Dist.
Gemma 3 12B IT	0,9080	0,9600	0,8740	0,7780	0,8700	0,6747	0,5160	0,7300	0,4993
Qwen3 8B	0,8300	0,9040	0,8140	0,4040	0,5100	0,2520	0,1192	0,2141	0,0953
Phi-4	0,6018	0,8407	0,4860	0,2808	0,4212	0,1680	0,1187	0,2348	0,0747
Nemotron Nano 9B v2	0,7600	0,8460	0,7347	0,4840	0,5940	0,3567	0,0887	0,1734	0,0747
Llama 3.1 8B Instruct	0,7640	0,8940	0,7100	0,3000	0,4720	0,1893	0,0200	0,0620	0,0133

Kết quả cho thấy Gemma 3 12B IT dẫn đầu toàn diện trên cả ba cấp độ và ba chỉ số đánh giá. Ở Cấp 1, hầu hết các mô hình đạt kết quả khá tốt với Acceptance trên 0,60 và Alignment trên 0,84, cho thấy các sLLM có khả năng sinh câu hỏi ở mức nhận thức thấp một cách đáng tin cậy.

Tuy nhiên, xu hướng sụt giảm chất lượng khi cấp độ nhận thức tăng là rất rõ

rệt. Từ Cấp 1 đến Cấp 3, Acceptance của Qwen3 8B giảm từ 0,8300 xuống 0,1192 (giảm 85,6%), Nemotron Nano 9B v2 giảm từ 0,7600 xuống 0,0887 (giảm 88,3%), và Llama 3.1 8B Instruct giảm từ 0,7640 xuống 0,0200 (giảm 97,4%). Đặc biệt, Llama 3.1 8B Instruct gần như thất bại hoàn toàn ở Cấp 3 với Distractor Quality chỉ đạt 0,0133, cho thấy mô hình này không có khả năng tạo ra các phương án nhiều có chất lượng cho câu hỏi phân tích.

Kết quả này xác nhận giả thuyết rằng các sLLM khi hoạt động độc lập gặp khó khăn lớn trong việc sinh câu hỏi đòi hỏi tư duy bậc cao, đặc biệt là ở cấp độ Phân tích. Điều này tạo động lực mạnh mẽ cho việc áp dụng kiến trúc đa tác tử nhằm bù đắp hạn chế nội tại của từng mô hình đơn lẻ.

5.1.2 Kết quả nhóm sLLM kết hợp đa tác tử

Bảng 5.2 trình bày kết quả khi cùng năm mô hình sLLM được tích hợp vào hệ thống đa tác tử, nơi các tác tử *Phân loại*, *Học sinh* và *Hiệu chỉnh* cùng phối hợp để nâng cao chất lượng câu hỏi.

Bảng 5.2: Kết quả đánh giá nhóm sLLM tích hợp hệ thống đa tác tử trên bộ dữ liệu RACE-High

Mô hình	Cấp 1 - Nhớ			Cấp 2 - Hiểu			Cấp 3 - Phân tích		
	Acc.	Align.	Dist.	Acc.	Align.	Dist.	Acc.	Align.	Dist.
Gemma 3 12B IT	0,9580	0,9940	0,9333	0,8380	0,9600	0,7800	0,7360	0,9920	0,6600
Qwen3 8B	0,9093	0,9897	0,8593	0,8577	0,9753	0,7107	0,5814	1,0000	0,5073
Phi-4	0,9753	0,9959	0,9407	0,9072	0,9711	0,7900	0,6845	1,0000	0,6187
Nemotron Nano 9B v2	0,9232	0,9778	0,8960	0,7818	0,9212	0,7033	0,5556	0,9818	0,4973
Llama 3.1 8B Instruct	0,8926	0,9853	0,8233	0,6863	0,9621	0,5453	0,4232	0,9979	0,3500

Kết quả cho thấy sự cải thiện ấn tượng và nhất quán trên toàn bộ năm mô hình khi được tích hợp vào hệ thống đa tác tử. Điểm nổi bật nhất là sự gia tăng vượt bậc ở chỉ số Alignment: hầu hết các mô hình đạt Alignment gần hoặc bằng 1,0 ở Cấp 2 và Cấp 3. Cụ thể, Qwen3 8B và Phi-4 đều đạt Alignment bằng 1,0000 ở Cấp 3, trong khi kết quả cơ sở tương ứng chỉ là 0,2141 và 0,2348. Điều này chứng tỏ tác tử *Phân loại* hoạt động rất hiệu quả trong việc đảm bảo câu hỏi sinh ra đúng cấp độ nhận thức yêu cầu.

Một phát hiện quan trọng là các mô hình có kết quả cơ sở yếu nhất lại hưởng lợi

nhiều nhất từ hệ thống đa tác tử. Phi-4, với Distractor Quality ở Cấp 1 chỉ đạt 0,4860 khi hoạt động độc lập, đã tăng lên 0,9407 khi kết hợp đa tác tử (cải thiện 93,6%). Llama 3.1 8B Instruct, mô hình có kết quả cơ sở thấp nhất, cũng ghi nhận mức cải thiện Acceptance ở Cấp 3 từ 0,0200 lên 0,4232 (tăng hơn 21 lần). Tuy nhiên, ngay cả với sự hỗ trợ của đa tác tử, Distractor Quality ở Cấp 3 của Llama vẫn chỉ đạt 0,3500, cho thấy hệ thống đa tác tử có thể bù đắp nhưng không thể hoàn toàn thay thế năng lực nội tại của mô hình nền.

5.1.3 Kết quả nhóm LLM thương mại

Bảng 5.3 trình bày kết quả của ba mô hình LLM thương mại, đóng vai trò đường cơ sở để đánh giá mức độ cạnh tranh của giải pháp sLLM kết hợp đa tác tử.

Bảng 5.3: Kết quả đánh giá nhóm LLM thương mại trên bộ dữ liệu RACE-High

Mô hình	Cấp 1 - Nhớ			Cấp 2 - Hiểu			Cấp 3 - Phân tích		
	Acc.	Align.	Dist.	Acc.	Align.	Dist.	Acc.	Align.	Dist.
Gemini 3 Flash	0,9820	0,9900	0,9647	0,9280	0,9320	0,8980	0,7753	0,8138	0,8980
GPT-5 Mini	0,9800	0,9920	0,9733	0,8240	0,8320	0,7387	0,8448	0,9032	0,8047
Claude Haiku 4.5	0,9700	0,9900	0,9453	0,8160	0,8340	0,7273	0,6720	0,7540	0,6380

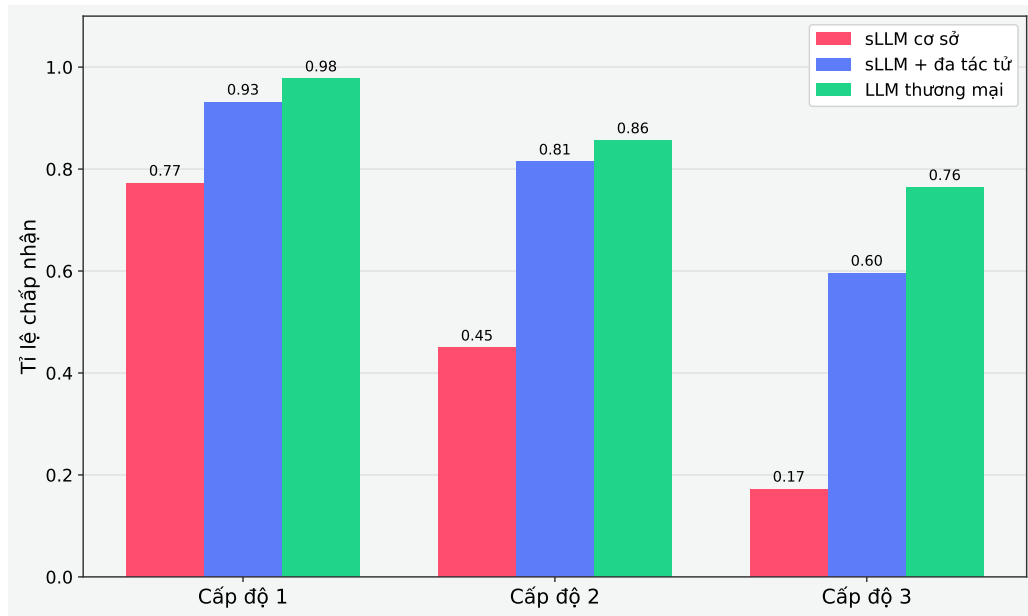
Nhóm LLM thương mại thể hiện chất lượng ổn định và đồng đều hơn so với nhóm sLLM cơ sở. Ở Cấp 1, cả ba mô hình đều đạt Acceptance trên 0,97 và Distractor Quality trên 0,94, cho thấy năng lực vượt trội trong việc sinh câu hỏi cấp thấp.

Gemini 3 Flash dẫn đầu tổng thể với Acceptance cao nhất ở Cấp 1 (0,9820) và Cấp 2 (0,9280), đồng thời đạt Distractor Quality xuất sắc ở Cấp 3 (0,8980). Đáng chú ý, GPT-5 Mini thể hiện sức mạnh đặc biệt ở Cấp 3 với Acceptance đạt 0,8448 - cao nhất trong toàn bộ nhóm thương mại, cho thấy khả năng sinh câu hỏi phân tích vượt trội. Claude Haiku 4.5, mặc dù là mô hình nhỏ nhất trong nhóm, vẫn đạt kết quả cạnh tranh với Acceptance ở Cấp 1 là 0,9700 và Distractor Quality ở Cấp 2 là 0,7273.

5.1.4 So sánh tổng hợp

Hình 5.1 trình bày biểu đồ so sánh chỉ số Acceptance trung bình giữa ba nhóm mô hình theo từng cấp độ nhận thức, cho phép nhận diện rõ ràng sự khác biệt về

năng lực giữa các nhóm.



Hình 5.1: So sánh chỉ số Acceptance trung bình giữa ba nhóm mô hình (sLLM cơ sở, sLLM kết hợp MAS, LLM thương mại) theo ba cấp độ nhận thức Bloom

Kết quả so sánh cho thấy ba phát hiện chính. Thứ nhất, ở Cấp 1 và Cấp 2, nhóm sLLM kết hợp đa tác tử đạt kết quả cạnh tranh sát sao với nhóm LLM thương mại. Cụ thể, Phi-4 kết hợp đa tác tử đạt Acceptance 0,9753 ở Cấp 1 và 0,9072 ở Cấp 2, trong khi Gemini 3 Flash đạt 0,9820 và 0,9280 tương ứng - chênh lệch không đáng kể. Thứ hai, ở Cấp 3, nhóm LLM thương mại vẫn duy trì ưu thế rõ rệt, đặc biệt ở chỉ số Distractor Quality, nơi Gemini 3 Flash đạt 0,8980 trong khi Gemma 3 12B IT kết hợp đa tác tử chỉ đạt 0,6600. Thứ ba, khoảng cách chất lượng giữa nhóm sLLM cơ sở và hai nhóm còn lại là rất lớn ở Cấp 2 và Cấp 3, khẳng định vai trò thiết yếu của kiến trúc đa tác tử hoặc năng lực mô hình lớn trong việc sinh câu hỏi tư duy bậc cao.

5.2 Kết quả OCR

Phần này đánh giá khả năng OCR của bảy mô hình: bốn mô hình thị giác-ngôn ngữ quy mô nhỏ chuyên dụng và ba mô hình ngôn ngữ lớn thương mại đa phương thức, trên hai điều kiện: ảnh sạch (chất lượng cao, không nhiễu) và ảnh nhiễu (đã qua tăng cường dữ liệu mô phỏng điều kiện thực tế). Hai chỉ số đánh giá được sử dụng là tỷ lệ lỗi ký tự (CER) và tỷ lệ lỗi từ (WER), trong đó giá trị càng thấp thể

hiện chất lượng OCR càng tốt.

5.2.1 Kết quả trên ảnh sạch

Bảng 5.4 trình bày kết quả OCR trên tập ảnh sạch, nơi văn bản được chụp trong điều kiện lý tưởng với độ phân giải cao và không có nhiễu.

Bảng 5.4: Kết quả OCR trên ảnh sạch

Mô hình	CER	WER
DeepSeek OCR 2	0,0681	0,0948
DotOCR 1.5	0,0075	0,0172
MinerU 2.5	0,0252	0,0427
PaddleOCR VL 1.5	0,0113	0,0232
Gemini 3 Flash	0,0084	0,0141
GPT-5 Mini	0,0150	0,0209
Claude Haiku 4.5	0,0238	0,0337

Trên ảnh sạch, DotOCR 1.5 đạt kết quả tốt nhất trong nhóm mô hình nhỏ gọn với CER chỉ 0,0075 và WER 0,0172, tương đương với hơn 99% ký tự được nhận dạng chính xác. Gemini 3 Flash đạt kết quả gần tương đương với CER 0,0084 và WER 0,0141, là mô hình thương mại có hiệu suất OCR tốt nhất. PaddleOCR VL 1.5 xếp thứ ba với CER 0,0113. Trong nhóm thương mại, GPT-5 Mini (CER 0,0150) và Claude Haiku 4.5 (CER 0,0238) đều đạt kết quả tốt, trong khi DeepSeek OCR 2 có CER cao nhất (0,0681). Nhìn chung, trên ảnh sạch, khoảng cách giữa các mô hình nhỏ gọn tốt nhất và mô hình thương mại là không đáng kể.

5.2.2 Kết quả trên ảnh nhiễu

Bảng 5.5 trình bày kết quả OCR trên tập ảnh đã qua tăng cường dữ liệu (data augmentation), mô phỏng các điều kiện chụp ảnh thực tế như nghiêng, mờ, nhiễu hạt và thay đổi ánh sáng.

Kết quả trên ảnh nhiễu cho thấy sự phân hóa rõ rệt giữa hai nhóm mô hình. Gemini 3 Flash thể hiện khả năng kháng nhiễu vượt trội nhất: CER chỉ tăng từ 0,0084 lên 0,0204 (tăng 2,4 lần), trong khi các mô hình nhỏ gọn tốt nhất tăng từ 10 đến 20

Bảng 5.5: Kết quả OCR trên ảnh nhiễu

Mô hình	CER	WER
DeepSeek OCR 2	0,1158	0,1579
DotOCR 1.5	0,1467	0,1771
MinerU 2.5	0,1079	0,2769
PaddleOCR VL 1.5	0,1143	0,1473
Gemini 3 Flash	0,0204	0,0326
GPT-5 Mini	0,0269	0,0409
Claude Haiku 4.5	0,1799	0,1932

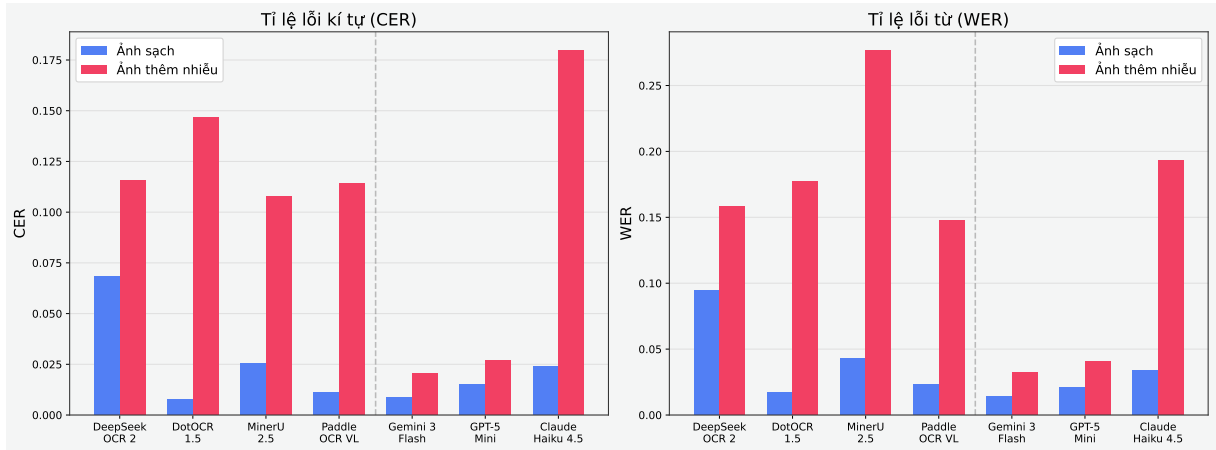
lần. GPT-5 Mini cũng duy trì hiệu suất ổn định với CER 0,0269 trên ảnh nhiễu. Hai mô hình thương mại này vượt trội hẳn so với toàn bộ nhóm mô hình nhỏ gọn trong điều kiện nhiễu.

Trong nhóm mô hình nhỏ gọn, MinerU 2.5 đạt CER thấp nhất (0,1079) nhưng lại có WER cao nhất (0,2769), cho thấy mô hình này nhận dạng ký tự tương đối tốt nhưng gặp khó khăn trong việc ghép từ chính xác. PaddleOCR VL 1.5 thể hiện sự cân bằng tốt nhất với cả CER (0,1143) và WER (0,1473) đều ở mức trung bình thấp. DotOCR 1.5, mô hình tốt nhất trên ảnh sạch, lại chịu mức suy giảm lớn nhất: CER tăng gần 20 lần từ 0,0075 lên 0,1467.

Đáng chú ý, Claude Haiku 4.5 là ngoại lệ trong nhóm thương mại: CER tăng từ 0,0238 lên 0,1799 (tăng 7,6 lần), ngang mức suy giảm của các mô hình nhỏ gọn. Kết quả này cho thấy khả năng kháng nhiễu không hoàn toàn tương quan với quy mô mô hình, mà phụ thuộc vào chiến lược huấn luyện và dữ liệu tiền xử lý của từng kiến trúc. Hình 5.2 so sánh trực quan hiệu suất OCR giữa hai điều kiện ảnh, cho thấy rõ mức độ suy giảm khi chuyển từ ảnh sạch sang ảnh nhiễu.

5.3 Phân tích chi phí

Bên cạnh chất lượng, chi phí sử dụng API là một yếu tố quan trọng trong việc lựa chọn giải pháp triển khai thực tế. Bảng 5.6 trình bày chi phí trung bình cho việc xử lý 100 đoạn văn mẫu (tương đương tối đa 1.500 câu hỏi) của từng cấu hình mô hình, được theo dõi thông qua OpenRouter API.

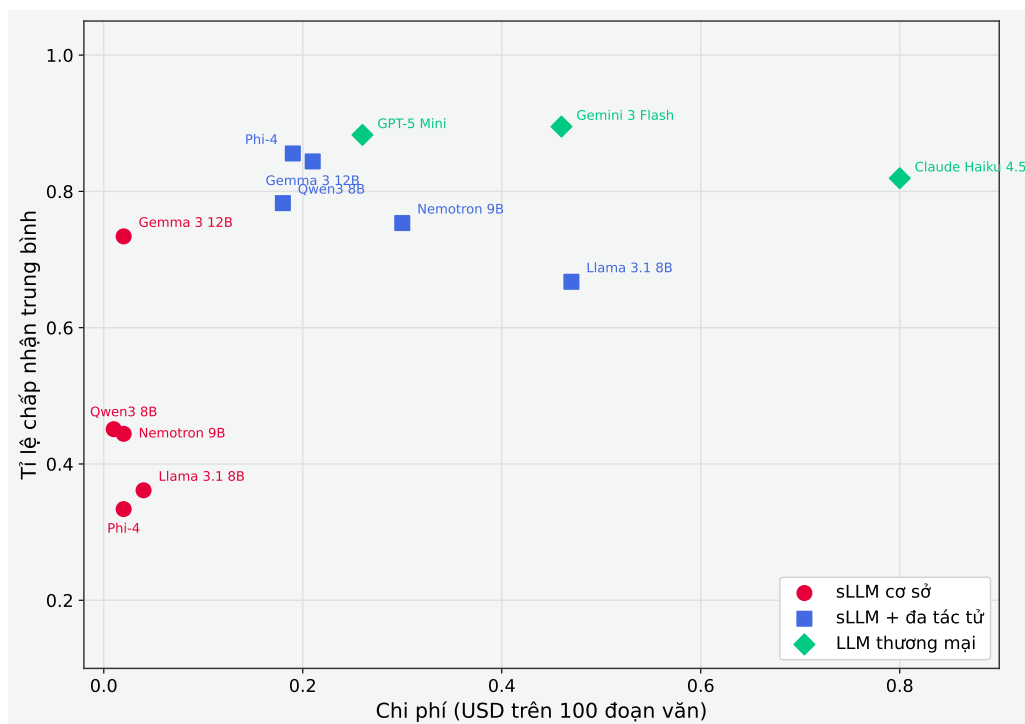


Hình 5.2: So sánh CER và WER giữa điều kiện ảnh sạch và ảnh nhiễu cho bảy mô hình OCR, minh họa mức độ suy giảm hiệu suất khi chuyển sang điều kiện thực tế

Bảng 5.6: Chi phí sử dụng API cho 100 đoạn văn mẫu (USD)

Nhóm	Mô hình	Chi phí (USD)
sLLM cơ sở	Gemma 3 12B IT	\$0,02
	Qwen3 8B	\$0,01
	Phi-4	\$0,02
	Nemotron Nano 9B v2	\$0,02
	Llama 3.1 8B Instruct	\$0,04
sLLM + đa tác tử	Gemma 3 12B IT	\$0,21
	Qwen3 8B	\$0,18
	Phi-4	\$0,19
	Nemotron Nano 9B v2	\$0,30
	Llama 3.1 8B Instruct	\$0,47
LLM thương mại	Gemini 3 Flash	\$0,46
	GPT-5 Mini	\$0,26
	Claude Haiku 4.5	\$0,80

Hình 5.3 trình bày biểu đồ phân tích mối quan hệ giữa chi phí và chất lượng (đo bằng Acceptance trung bình trên ba cấp độ) cho từng cấu hình mô hình.



Hình 5.3: Biểu đồ phân tán chi phí API (USD) so với Acceptance trung bình trên ba cấp độ nhận thức, xác định biên Pareto tối ưu giữa chất lượng và ngân sách

Phân tích chi phí cho thấy ba điểm đáng chú ý. Thứ nhất, nhóm sLLM cơ sở có chi phí cực thấp (từ \$0,01 đến \$0,04 cho 100 đoạn văn), tuy nhiên chất lượng ở Cấp 2 và Cấp 3 không đáp ứng được yêu cầu thực tế. Thứ hai, hệ thống đa tác tử làm tăng chi phí khoảng 10 đến 20 lần so với sLLM cơ sở (ví dụ: Gemma từ \$0,02 lên \$0,21, Qwen3 từ \$0,01 lên \$0,18), nhưng mức cải thiện chất lượng thu được là vượt trội - đặc biệt ở Cấp 2 và Cấp 3 - tạo ra tỷ suất lợi ích trên chi phí rất cao. Thứ ba, so với nhóm LLM thương mại, cấu hình Gemma 3 12B IT kết hợp đa tác tử nổi lên như lựa chọn tối ưu theo biên Pareto: chỉ với \$0,21 (bằng 45,7% chi phí của Gemini 3 Flash và 26,3% chi phí của Claude Haiku 4.5), cấu hình này đạt chất lượng cạnh tranh ở Cấp 1 và Cấp 2, đồng thời duy trì kết quả chấp nhận được ở Cấp 3.

5.4 Thảo luận

Từ kết quả thực nghiệm được trình bày ở các phần trên, nghiên cứu rút ra sáu nhận định chính.

Hệ thống đa tác tử giúp tăng chất lượng cho sLLM. Kết quả cho thấy kiến trúc đa tác tử không chỉ cải thiện chất lượng mà còn thể hiện một đặc tính quan trọng: các mô hình có kết quả cơ sở yếu nhất lại hưởng lợi nhiều nhất. Phi-4, với Distractor Quality ở Cấp 1 chỉ đạt 0,4860 khi hoạt động đơn lẻ, đã tăng lên 0,9407 khi kết hợp đa tác tử - mức cải thiện 93,6%. Llama 3.1 8B Instruct ghi nhận cải thiện Acceptance ở Cấp 3 hơn 21 lần. Điều này gợi ý rằng hệ thống đa tác tử hoạt động hiệu quả nhất khi mô hình nền có năng lực tiềm ẩn chưa được khai thác đầy đủ trong chế độ hoạt động đơn lẻ.

Tác tử *Phân loại* hoạt động hiệu quả nhờ cơ chế chống thiên lệch. Chỉ số Alignment đạt gần hoặc bằng 1,0 ở Cấp 2 và Cấp 3 cho hầu hết các mô hình trong nhóm đa tác tử (Qwen3 8B và Phi-4 đạt 1,0000 ở Cấp 3), chứng tỏ việc xáo trộn ngẫu nhiên câu hỏi trước khi phân loại giúp đảm bảo tính khách quan trong quá trình phân loại câu hỏi theo thang nhận thức Bloom.

Tác tử *Học sinh* đóng vai trò lọc câu hỏi mơ hồ. Bằng cách mô phỏng quá trình trả lời của học sinh, tác tử này giúp phát hiện các câu hỏi có đề bài thiếu rõ ràng, phương án nhiễu quá dễ loại trừ, hoặc đáp án không phân biệt rõ với các phương án sai. Cơ chế phản hồi từ tác tử *Học sinh* đến tác tử *Hiệu chỉnh* tạo thành vòng lặp cải tiến liên tục, giúp nâng cao chất lượng câu hỏi qua nhiều vòng.

Biên Pareto: Gemma 3 12B IT kết hợp đa tác tử là lựa chọn giá trị tối ưu. Khi xem xét đồng thời chi phí và chất lượng, cấu hình Gemma 3 12B IT với hệ thống đa tác tử nổi lên là điểm tối ưu trên biên Pareto. Với chi phí chỉ \$0,21 cho 100 đoạn văn, cấu hình này đạt Acceptance 0,9580 (Cấp 1), 0,8380 (Cấp 2) và 0,7360 (Cấp 3), vượt trội so với Claude Haiku 4.5 (\$0,80) ở cả ba cấp độ và cạnh tranh sát sao với Gemini 3 Flash (\$0,46) ở Cấp 1 và Cấp 2.

OCR: mô hình nhỏ gọn gặp khó khăn với ảnh nhiễu, mô hình thương mại phân hóa rõ rệt. Kết quả OCR cho thấy dù các mô hình nhỏ gọn hoạt động tốt trên ảnh sạch (DotOCR 1.5 đạt CER 0,0075), hiệu suất suy giảm đáng kể trên ảnh nhiễu (CER tăng lên 0,1079-0,1467). Trong khi đó, Gemini 3 Flash và GPT-5 Mini duy trì hiệu suất ổn định trên ảnh nhiễu (CER lần lượt 0,0204 và 0,0269),

nhưng Claude Haiku 4.5 lại chịu mức suy giảm tương đương nhóm mô hình nhỏ gọn (CER 0,1799). Phát hiện này nhấn mạnh rằng khả năng kháng nhiễu phụ thuộc vào chiến lược huấn luyện chứ không hoàn toàn vào quy mô mô hình, đồng thời nhấn mạnh tầm quan trọng của bước tiền xử lý ảnh trong đường ống xử lý thực tế.

Hạn chế của phương pháp đánh giá LLM-as-a-Judge. Toàn bộ kết quả đánh giá trong nghiên cứu này được thực hiện bằng phương pháp G-Eval, sử dụng mô hình ngôn ngữ lớn làm người đánh giá tự động. Mặc dù phương pháp này cho phép đánh giá quy mô lớn với chi phí thấp và tính nhất quán cao, nó vẫn thiếu sự đối chiếu với đánh giá của con người - tiêu chuẩn vàng trong đánh giá chất lượng giáo dục. Các nghiên cứu trước đã chỉ ra rằng LLM-as-a-Judge có thể mắc các thiên lệch như ưu tiên câu trả lời dài hơn hoặc thiên vị đầu ra của chính mình. Do đó, những kết quả định lượng trong nghiên cứu này cần phải được diễn giải với sự thận trọng nhất định.

Chương 6

Thiết kế và hiện thực ứng dụng Openlet

Chương này trình bày cách thức hiện thực hóa mô hình lý thuyết từ Chương 3 vào ứng dụng web Openlet thực tế. Nội dung bao gồm phân tích các ca sử dụng, thiết kế kiến trúc toàn diện và trình bày các luồng hoạt động chính để đáp ứng nhu cầu tương tác của người dùng cuối.

6.1 Giới thiệu ứng dụng và phạm vi sử dụng

Chương 3 của khóa luận đã trình bày các thành phần nghiên cứu cốt lõi của hệ thống Openlet, bao gồm quy trình nhận dạng văn bản từ ảnh, cơ chế sinh câu hỏi trắc nghiệm và khung đánh giá chất lượng. Trên cơ sở đó, chương này tập trung trình bày cách các kết quả nghiên cứu được hiện thực hóa thành một ứng dụng Web hoàn chỉnh, có khả năng phục vụ trực tiếp cho hoạt động tạo bài kiểm tra và làm bài trực tuyến trong môi trường giáo dục.

Openlet hướng tới hai nhóm người dùng chính. Nhóm thứ nhất là giáo viên hoặc người biên soạn học liệu, những người cần chuyển nhanh tài liệu học tập thành bài kiểm tra có thể chỉnh sửa và công bố. Nhóm thứ hai là người làm bài, những người truy cập bài kiểm tra qua liên kết chia sẻ, thực hiện bài làm và nhận phản hồi theo cấu hình mà giáo viên đã đặt ra.

Trong phạm vi khóa luận, Openlet tập trung vào một chu trình nghiệp vụ khép kín gồm tiếp nhận học liệu đầu vào, sinh câu hỏi tự động, biên tập nội dung, công bố bài kiểm tra, tổ chức làm bài và theo dõi kết quả. Ứng dụng hỗ trợ ba dạng đầu vào là ảnh chụp tài liệu, tệp PDF và văn bản thuần. Tuy nhiên, hệ thống không hướng tới thay thế toàn bộ một hệ quản lý học tập hoàn chỉnh, mà tập trung vào bài toán chuyển đổi học liệu thành bài kiểm tra trực tuyến một cách nhanh, linh hoạt và có kiểm soát chất lượng.

6.2 Công nghệ sử dụng

6.2.1 Nhóm công nghệ giao diện

Lớp giao diện của Openlet được xây dựng bằng **Next.js** [31] kết hợp với **React** [18]. Cách lựa chọn này cho phép hệ thống tổ chức giao diện theo các trang chức năng rõ ràng, đồng thời hỗ trợ tốt việc cập nhật dữ liệu theo thời gian thực khi trạng thái xử lý của bài kiểm tra thay đổi. Bên cạnh đó, hệ thống thành phần giao diện được hoàn thiện bằng các thư viện hỗ trợ tương tác, biểu mẫu và hiển thị, giúp rút ngắn thời gian phát triển mà vẫn đảm bảo tính nhất quán trong trải nghiệm sử dụng.

6.2.2 Nhóm công nghệ dịch vụ và lưu trữ

Phần dịch vụ của Openlet được triển khai trên nền tảng **Firebase** [10]. Hệ thống sử dụng cơ chế xác thực để quản lý danh tính người dùng, nền tảng lưu trữ dữ liệu để quản lý thông tin bài kiểm tra và kết quả làm bài, vùng lưu trữ tệp để tiếp nhận ảnh và tệp PDF, cùng các hàm xử lý phía máy chủ để thực hiện những nghiệp vụ nhạy cảm như nhận dạng văn bản, sinh câu hỏi, truy xuất đề công khai, chấm điểm và cập nhật thống kê.

Việc sử dụng hạ tầng không máy chủ giúp Openlet phù hợp với phạm vi của một khóa luận tốt nghiệp. Hệ thống không cần đầu tư riêng một máy chủ ứng dụng độc lập nhưng vẫn duy trì được khả năng triển khai nhanh, đơn giản hóa khâu vận hành và giữ được sự tách bạch hợp lý giữa mặt giao diện, xử lý nghiệp vụ và lưu trữ dữ liệu.

6.2.3 Nhóm công nghệ mô hình trí tuệ nhân tạo

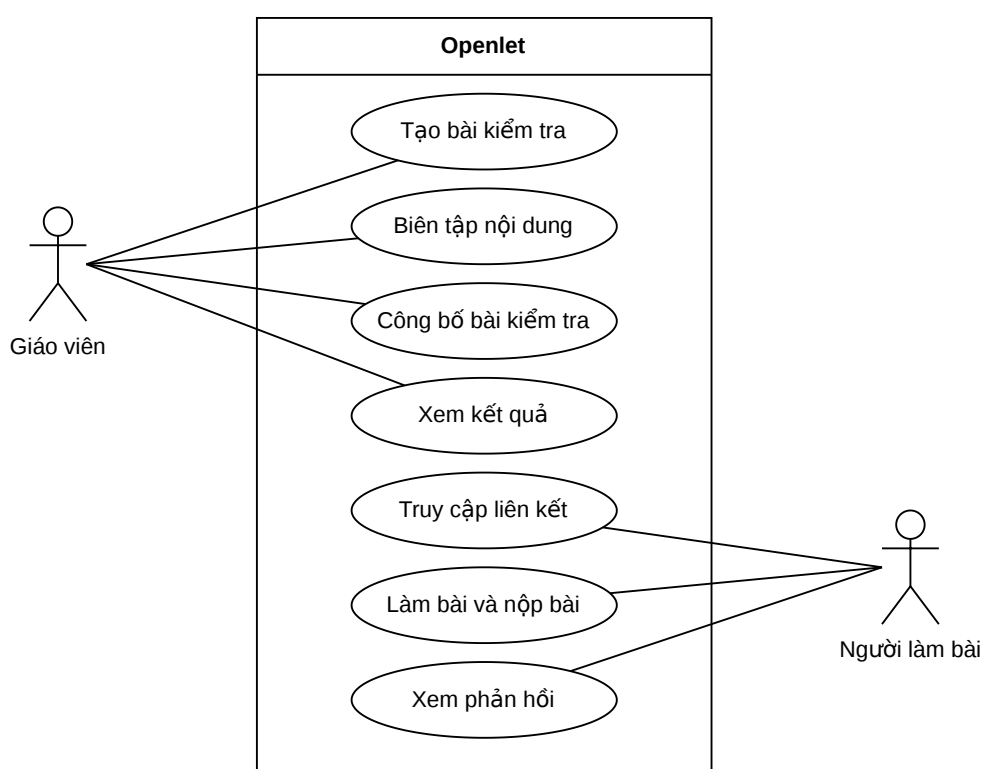
Đối với các tác vụ liên quan đến trí tuệ nhân tạo, Openlet sử dụng **OpenRouter** [25] như một cổng trung gian để truy cập các mô hình nhận dạng văn bản và mô hình ngôn ngữ. Cách triển khai này giúp hệ thống có thể thay đổi mô hình theo từng mục đích sử dụng mà không phải điều chỉnh lại kiến trúc chung của ứng dụng. Trên thực tế, Openlet hỗ trợ cả mô hình thương mại lẫn mô hình mã nguồn mở, qua đó phù hợp với nhiều cấu hình vận hành khác nhau.

6.3 Phân tích chức năng và tác nhân sử dụng

Openlet được tổ chức xoay quanh hai tác nhân sử dụng chính:

- **Giáo viên:** Khởi tạo bài kiểm tra, nhập nội dung từ tài liệu hoặc văn bản, lựa chọn cấu hình sinh câu hỏi, biên tập câu hỏi, công bố bài kiểm tra, thiết lập chính sách làm bài và theo dõi kết quả.
- **Người làm bài:** Truy cập bài kiểm tra qua liên kết, đăng nhập ẩn danh hoặc bằng tài khoản khi cần thiết, thực hiện bài làm, nộp đáp án và xem kết quả trong phạm vi cho phép.

Để khái quát phạm vi chức năng của ứng dụng, sơ đồ ca sử dụng tổng quát của Openlet được trình bày ở Hình 6.1.



Hình 6.1: Biểu đồ ca sử dụng tổng quát của Openlet với hai tác nhân chính: giáo viên (quản lý vòng đời bài kiểm tra) và người làm bài (truy cập và nộp bài)

Hình 6.1 cho thấy giáo viên là tác nhân khởi tạo gần như toàn bộ vòng đời của một bài kiểm tra. Từ giao diện biên tập, giáo viên có thể nhập tài liệu đầu vào,

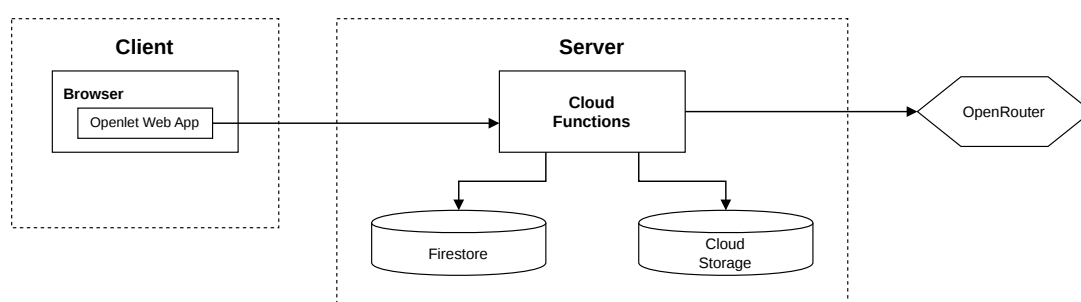
yêu cầu hệ thống sinh câu hỏi tự động, rà soát và sửa nội dung, công bố bài kiểm tra, cấu hình mức hiển thị kết quả, cho phép hoặc không cho phép làm lại, đồng thời theo dõi số liệu tổng hợp và danh sách lượt làm bài. Ở chiều ngược lại, người làm bài chỉ tương tác với phần chức năng công khai của hệ thống, bao gồm truy cập bài kiểm tra, thực hiện bài làm, nộp bài và xem kết quả theo chính sách công bố.

Việc rút gọn hệ thống về hai tác nhân chính giúp mô hình phân quyền của Openlet rõ ràng và phù hợp với quy mô của một ứng dụng phục vụ lớp học hoặc nhóm học nhỏ. Thay vì đưa vào nhiều vai trò trung gian như quản trị hệ thống, trợ giảng hoặc người kiểm duyệt nội dung, thiết kế hiện tại tập trung vào đúng hai nhóm người dùng tham gia trực tiếp vào giá trị cốt lõi của sản phẩm. Cách tổ chức này làm giảm độ phức tạp của giao diện, đơn giản hóa luật bảo mật và giúp người sử dụng dễ hình dung trách nhiệm của mình trong quy trình vận hành.

6.4 Thiết kế hệ thống

6.4.1 Kiến trúc tổng thể

Sơ đồ kiến trúc tổng thể của ứng dụng được trình bày ở Hình 6.2 để thể hiện cách Openlet kết nối lớp giao diện, lớp dịch vụ và lớp mô hình trí tuệ nhân tạo.



Hình 6.2: Kiến trúc ba tầng của Openlet: tầng giao diện (Next.js/React), tầng dịch vụ (Firebase), và tầng AI (OpenRouter kết nối các mô hình OCR/LLM)

Openlet được triển khai theo kiến trúc ba tầng. Tầng giao diện là ứng dụng web chạy trong trình duyệt. Tầng dịch vụ được đặt trên nền tảng Firebase, bao gồm cơ chế xác thực, nền tảng lưu trữ dữ liệu, vùng lưu trữ tệp và các hàm xử lý phía máy chủ. Tầng trí tuệ nhân tạo nằm ở lớp ngoài cùng, nơi các mô hình nhận dạng văn bản và mô hình ngôn ngữ được gọi thông qua OpenRouter để phục vụ các tác

vụ như trích xuất văn bản, phân tích nội dung, sinh câu hỏi và sinh giải thích.

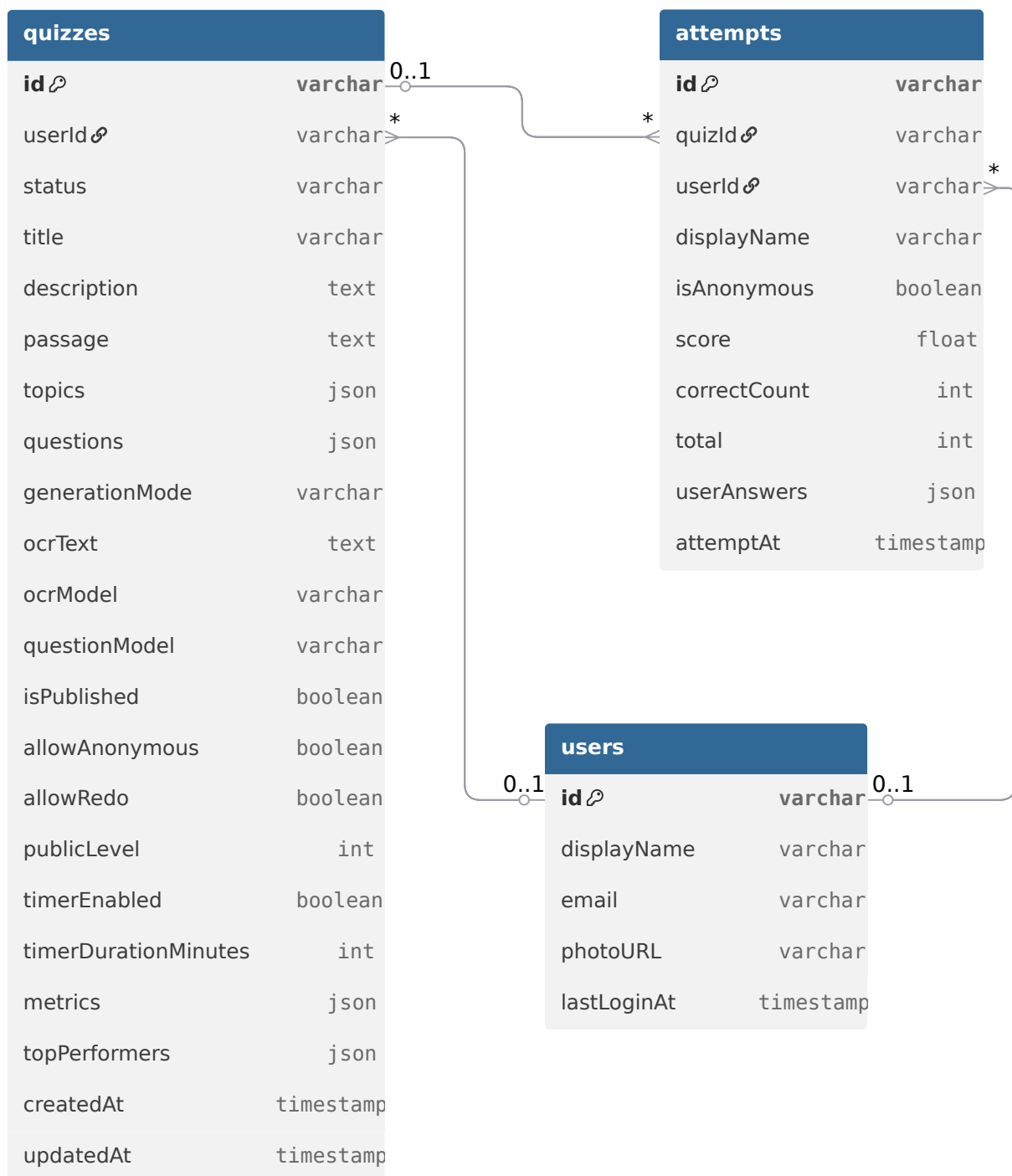
Trong quá trình vận hành, giao diện Web kết nối trực tiếp với Firebase Authentication để xác thực người dùng, với Cloud Firestore để đọc ghi dữ liệu và đồng bộ trạng thái theo thời gian thực, đồng thời dùng Cloud Storage để tải các tệp đầu vào. Riêng những tác vụ nhạy cảm hoặc cần xử lý nền như truy xuất đề công khai, chấm điểm, nhận dạng văn bản, sinh câu hỏi và cập nhật thống kê được dồn vào Cloud Functions. Khối Cloud Functions vì vậy đóng vai trò đầu mối xử lý phía máy chủ, vừa làm việc với Firestore và Storage, vừa gọi OpenRouter khi cần dùng mô hình trí tuệ nhân tạo. Tầng trí tuệ nhân tạo không giao tiếp trực tiếp với người dùng cuối mà chỉ được gọi từ lớp xử lý này.

Openlet không lựa chọn kiến trúc vi dịch vụ đầy đủ vì phạm vi nghiệp vụ của sản phẩm tập trung vào một chu trình thống nhất. Kiến trúc ba tầng trên nền tảng không máy chủ đảm bảo được sự tách bạch trách nhiệm giữa giao diện, xử lý nghiệp vụ và mô hình trí tuệ nhân tạo, đồng thời giúp rút ngắn thời gian triển khai và giảm chi phí vận hành.

6.4.2 Mô hình dữ liệu

Mô hình dữ liệu của Openlet được tổ chức quanh ba bảng thông tin quan trọng. Bảng **users** lưu hồ sơ của người đăng nhập bằng tài khoản Google, bao gồm định danh người dùng, tên hiển thị, thư điện tử, ảnh đại diện và thời điểm đăng nhập gần nhất. Bảng **quizzes** là trung tâm của toàn bộ nghiệp vụ, lưu trạng thái xử lý, tiêu đề, mô tả, đoạn văn nguồn, danh sách câu hỏi, chế độ sinh câu hỏi, cấu hình mô hình, thiết lập công bố, cấu hình bộ đếm thời gian, số liệu thống kê tổng hợp, bảng xếp hạng người làm bài và các mốc thời gian tạo, cập nhật. Bảng **attempts** gắn với từng bài kiểm tra để lưu thông tin người tham gia, tên hiển thị, trạng thái ẩn danh, điểm số, số câu đúng, tổng số câu hỏi, đáp án đã chọn và thời điểm nộp bài. Sơ đồ dữ liệu chính của ứng dụng được trình bày ở Hình 6.3.

Ngoài những dữ liệu phục vụ hiển thị, bảng **quizzes** còn lưu các thông tin phục vụ xử lý nền như văn bản đã nhận dạng từ tài liệu, lựa chọn mô hình nhận dạng văn bản, lựa chọn mô hình sinh câu hỏi và các chỉ số tổng hợp. Những trường này giúp hệ thống có thể theo dõi và tái hiện đúng ngữ cảnh xử lý khi cần, đồng thời hỗ trợ giáo viên quan sát tình trạng xử lý và kết quả sử dụng trên cùng một bản



Hình 6.3: Mô hình dữ liệu chính của Openlet gồm ba bảng: **users** (hồ sơ người dùng), **quizzes** (bài kiểm tra và câu hỏi), và **attempts** (lượt làm bài và kết quả)

ghi chính.

Cloud Firestore được lựa chọn vì cấu trúc dữ liệu của Openlet có tính lồng nhau rõ rệt. Một bài kiểm tra không chỉ có thông tin mô tả đơn giản mà còn chứa danh sách câu hỏi, cấu hình hiển thị kết quả, thiết lập bộ đếm thời gian và các chỉ số tổng hợp. Với kiểu dữ liệu này, mô hình cơ sở dữ liệu dạng tài liệu phù hợp tự nhiên hơn so với cơ sở dữ liệu quan hệ vốn cần tách ra nhiều bảng và nhiều phép nối. Bên cạnh đó, nền tảng này hỗ trợ đồng bộ dữ liệu theo thời gian thực, rất phù hợp cho giao diện theo dõi tiến trình xử lý và giao diện xem kết quả.

Thiết kế tách **attempts** thành một bảng riêng gắn với từng bài kiểm tra giúp giới hạn kích thước của bản ghi chính và giữ cho thao tác đọc bài kiểm tra luôn nhẹ. Song song với đó, các số liệu tổng hợp như tổng số lượt làm, điểm trung bình, điểm cao nhất, điểm thấp nhất, phân bố điểm và danh sách người đạt thành tích cao nhất được lưu sẵn trong bản ghi **quizzes**. Cách làm này giúp giao diện của giáo viên có thể truy xuất nhanh các số liệu tổng quan mà không phải quét lại toàn bộ lịch sử nộp bài ở mỗi lần hiển thị.

6.4.3 Bảo mật và kiểm soát truy cập

Mô hình truy cập của Openlet được xây dựng xoay quanh hai nguyên tắc. Thứ nhất, giáo viên là chủ sở hữu toàn bộ dữ liệu gốc của bài kiểm tra, bao gồm đáp án đúng, giải thích và lịch sử làm bài. Thứ hai, người làm bài chỉ được tiếp cận phần dữ liệu tối thiểu cần thiết để thực hiện bài làm và nhận kết quả theo phạm vi mà giáo viên đã cấu hình.

Đối với giáo viên, ứng dụng sử dụng đăng nhập bằng tài khoản Google để tạo hồ sơ người dùng và gán quyền sở hữu cho từng bài kiểm tra. Đối với người làm bài, hệ thống hỗ trợ hai hình thức truy cập. Nếu giáo viên cho phép làm bài ẩn danh, người dùng chỉ cần nhập tên hiển thị để bắt đầu. Nếu giáo viên tắt chế độ ẩn danh, người dùng phải xác thực bằng tài khoản trước khi truy cập. Chính sách cho phép làm lại chỉ được áp dụng trong trường hợp người làm bài có tài khoản xác thực.

Một yêu cầu bảo mật quan trọng của Openlet là đáp án đúng không được gửi trực tiếp về giao diện công khai. Người làm bài không đọc dữ liệu bài kiểm tra từ nền tảng lưu trữ dữ liệu theo cách trực tiếp, mà phải đi qua hàm xử lý phía máy chủ. Tại đây, hệ thống chỉ trả về nội dung câu hỏi và các phương án lựa chọn, đồng thời

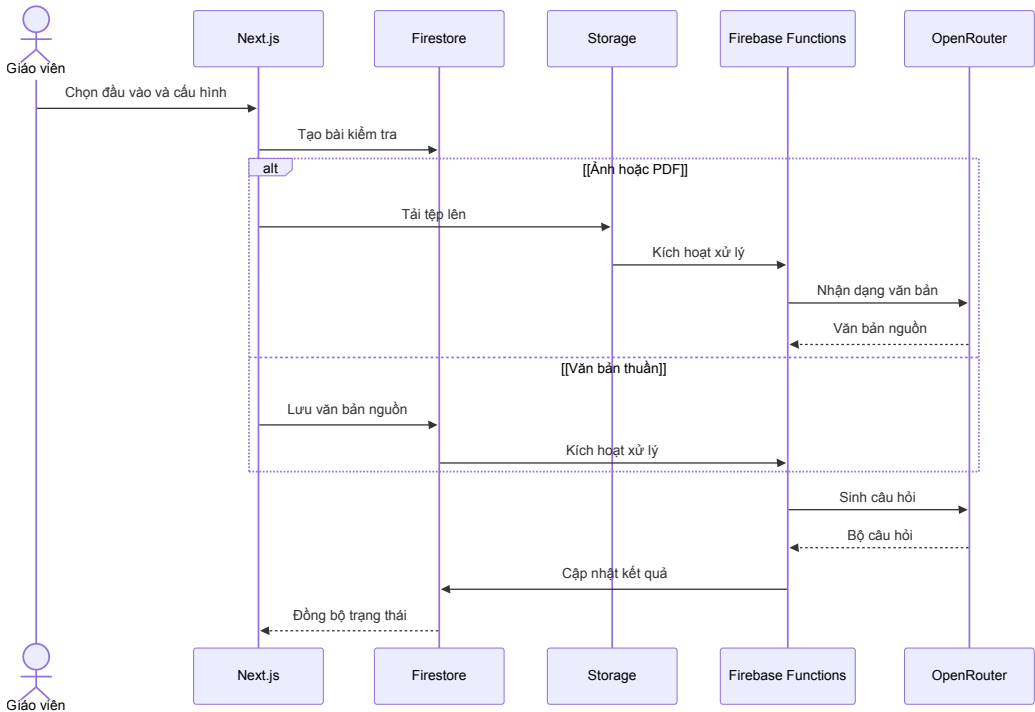
loại bỏ đáp án đúng và phần giải thích. Sau khi nộp bài, phản hồi trả về tiếp tục được lọc theo năm mức hiển thị kết quả, từ mức chỉ xác nhận đã nộp bài cho tới mức hiển thị đầy đủ điểm số, đúng sai, đáp án đúng và lời giải thích.

Thiết kế này giúp Openlet giải quyết đồng thời hai yêu cầu tưởng như mâu thuẫn: bài kiểm tra phải có thể chia sẻ công khai qua liên kết, nhưng đáp án đúng vẫn phải được bảo vệ. Nếu lưu toàn bộ dữ liệu bài kiểm tra ở phía giao diện và cho phép người làm bài tải trực tiếp, việc công bố liên kết sẽ kéo theo nguy cơ lộ đáp án. Bởi vậy, đáp án đúng được giữ lại ở tầng máy chủ, còn giao diện chỉ nhận phần dữ liệu đã được rút gọn theo đúng ngữ cảnh sử dụng.

6.5 Hiện thực các luồng chức năng cốt lõi

6.5.1 Luồng tạo bài kiểm tra tự động

Sơ đồ tuần tự cho chức năng tạo bài kiểm tra tự động được trình bày ở Hình 6.4 để minh họa cách giao diện, nền tảng lưu trữ dữ liệu, vùng lưu trữ tệp, hàm xử lý nền và lớp mô hình trí tuệ nhân tạo phối hợp với nhau.



Hình 6.4: Biểu đồ tuần tự của luồng tạo bài kiểm tra tự động: từ tiếp nhận tài liệu đầu vào, qua OCR và MAS sinh câu hỏi, đến cập nhật trạng thái theo thời gian thực

Khi giáo viên nhập nội dung bằng trí tuệ nhân tạo, quy trình bắt đầu từ giao diện biên tập bài kiểm tra. Giáo viên có thể chọn một trong ba dạng đầu vào là ảnh, tệp PDF hoặc văn bản thuần; đồng thời xác định mô hình nhận dạng văn bản, mô hình sinh câu hỏi, số lượng câu hỏi mong muốn và chế độ sinh câu hỏi.

Sau khi tiếp nhận yêu cầu, hệ thống khởi tạo một bản ghi xử lý để theo dõi tiến trình. Nếu đầu vào là ảnh hoặc tệp PDF, dữ liệu được tải tạm thời lên vùng lưu trữ, sau đó một hàm xử lý nền được kích hoạt để thực hiện nhận dạng văn bản. Riêng với đầu vào là văn bản thuần, hệ thống bỏ qua bước nhận dạng và chuyển thẳng sang giai đoạn sinh câu hỏi. Ở chế độ cơ bản, văn bản nguồn được đưa trực tiếp vào một lần sinh duy nhất. Ở chế độ đa tác tử, hệ thống bổ sung thêm bước phân tích nội dung trước khi sinh câu hỏi, kiểm chứng, hiệu chỉnh phương án và sinh giải thích.

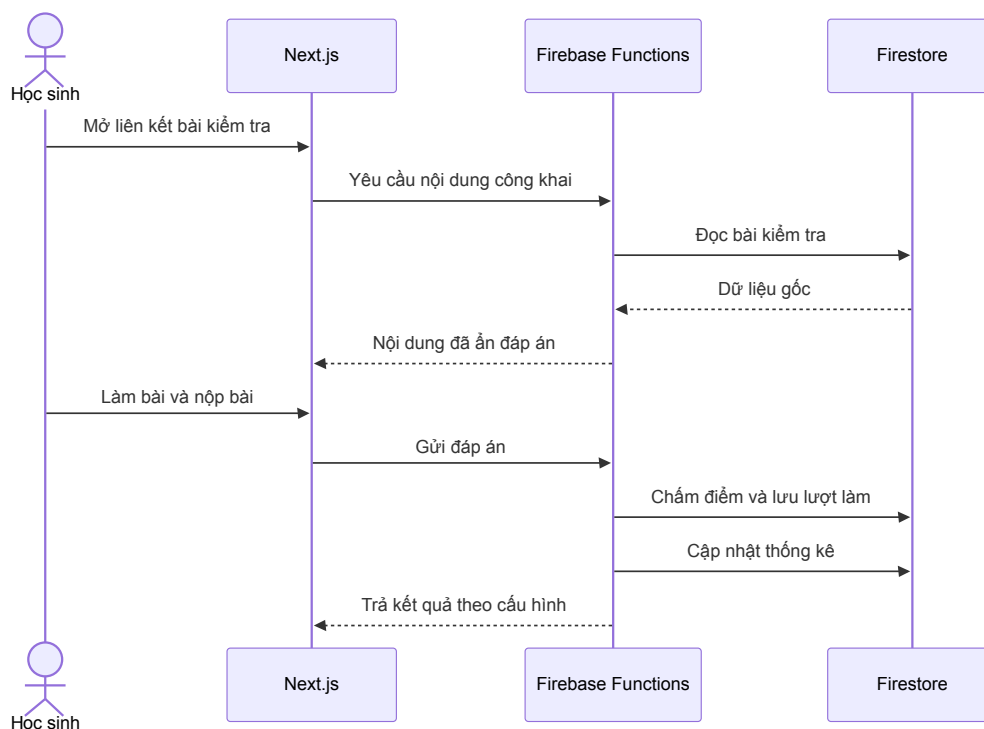
Trong toàn bộ quá trình này, trạng thái xử lý của bài kiểm tra được cập nhật liên tục trong nền tảng lưu trữ dữ liệu. Giao diện của giáo viên theo dõi các thay đổi đó để hiển thị tiến trình theo thời gian thực. Khi hoàn tất, hệ thống lưu bộ câu hỏi và các siêu dữ liệu được rút trích vào bản ghi bài kiểm tra, sau đó đưa dữ liệu về môi trường biên tập để giáo viên rà soát, chỉnh sửa và quyết định có công bố bài kiểm tra hay không.

Luồng xử lý theo trạng thái được lựa chọn vì nó phù hợp với bản chất bất đồng bộ của bài toán. Nhận dạng văn bản, sinh câu hỏi và sinh giải thích đều là các bước có độ trễ cao, không nên buộc giao diện phải chờ liên tục cho tới khi hoàn tất. Bằng cách chia quy trình thành các giai đoạn rõ ràng, hệ thống dễ theo dõi tiến trình, phát hiện lỗi và tránh kích hoạt lặp cùng một bước xử lý. Thiết kế này cũng cho phép tối ưu thời gian phản hồi: đầu vào văn bản thuần không phải đi qua bước nhận dạng, còn chế độ cơ bản có thể bỏ qua khâu phân tích nội dung để giảm độ trễ.

So với phiên bản nghiên cứu ở Chương 3, phiên bản ứng dụng chỉ giữ lại những vòng kiểm chứng thật sự cần thiết và giới hạn số lần lặp, phản ánh ưu tiên của một sản phẩm ứng dụng thực tế: giáo viên cần nhận kết quả đủ nhanh để tiếp tục biên tập, trong khi chất lượng vẫn được giữ ở mức chấp nhận được nhờ các lớp kiểm soát trọng yếu.

6.5.2 Luồng truy cập, làm bài và nộp bài

Sơ đồ tuần tự cho chức năng truy cập bài kiểm tra, làm bài và nộp bài được trình bày ở Hình 6.5 để làm rõ cách hệ thống bảo vệ đáp án đúng trong khi vẫn hỗ trợ chia sẻ công khai qua liên kết.



Hình 6.5: Biểu đồ tuần tự của luồng truy cập, làm bài và nộp bài: người làm bài nhận đề đã lọc đáp án, chấm điểm phía máy chủ, và nhận phản hồi theo mức cấu hình

Khi người làm bài mở liên kết công khai, giao diện trước hết kiểm tra chính sách truy cập của bài kiểm tra. Nếu giáo viên cho phép làm bài ẩn danh, người dùng có thể nhập tên hiển thị để tiếp tục. Nếu không cho phép ẩn danh, hệ thống yêu cầu người dùng xác thực bằng tài khoản Google. Trong trường hợp giáo viên không cho phép làm lại, hệ thống còn kiểm tra xem người dùng đã có lượt làm bài trước đó hay chưa.

Sau bước kiểm tra truy cập, giao diện gửi yêu cầu lấy đề tới hàm xử lý phía máy chủ. Hàm này đọc dữ liệu gốc của bài kiểm tra, xác minh trạng thái công bố và chỉ trả về phần dữ liệu đã được làm sạch, bao gồm nội dung câu hỏi và các phương án trả lời. Nhờ vậy, người làm bài có thể xem đề và thao tác làm bài nhưng không thể truy cập đáp án đúng thông qua dữ liệu được tải về.

Khi người làm bài nộp đáp án, hàm xử lý phía máy chủ đọc bộ câu hỏi gốc, thực

hiện chấm điểm, ghi lại lượt làm bài vào bảng tương ứng, đồng thời cập nhật các số liệu tổng hợp và bảng xếp hạng trong bản ghi bài kiểm tra. Phản hồi cuối cùng gửi về giao diện không phải là toàn bộ dữ liệu chấm điểm, mà là phiên bản đã được lọc theo mức hiển thị kết quả mà giáo viên cấu hình trước đó.

Đây là luồng thể hiện rõ nhất quyết định giữ đáp án đúng ở phía máy chủ. Nếu giao diện được phép nhận trực tiếp toàn bộ nội dung bài kiểm tra, việc công bố liên kết sẽ khiến người làm bài có thể khai thác dữ liệu để suy ra đáp án. Bởi vậy, hệ thống buộc phải tách thao tác truy xuất đề, thao tác chấm điểm và thao tác xem kết quả thành các bước có kiểm soát, trong đó mọi dữ liệu nhạy cảm chỉ tồn tại ở tầng dịch vụ.

Việc ghi lượt làm bài vào một bảng riêng cho từng bài kiểm tra giúp truy vết lịch sử làm bài rõ ràng và hỗ trợ quản lý từng lần nộp. Trong khi đó, việc cập nhật sẵn số liệu tổng hợp trong bản ghi bài kiểm tra giúp giao diện tổng quan của giáo viên hiển thị ngay các thống kê quan trọng như số lượt làm, phân bố điểm và nhóm người đạt kết quả cao mà không cần xử lý lại toàn bộ dữ liệu lịch sử ở mỗi lần mở trang.

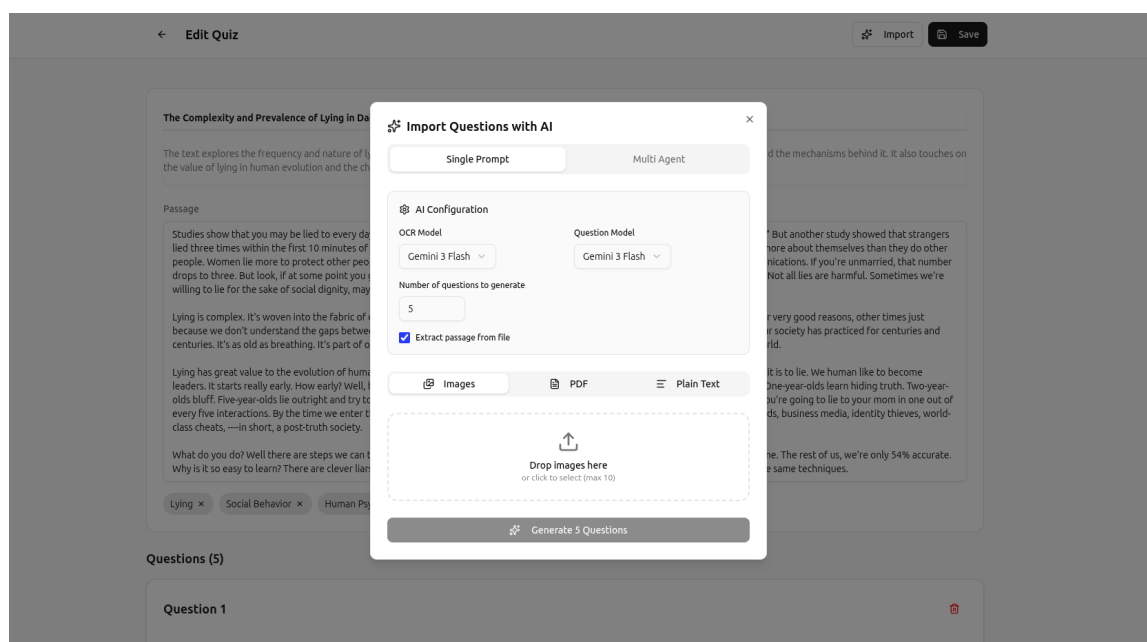
6.6 Giao diện tiêu biểu của ứng dụng

6.6.1 Giao diện dành cho giáo viên

Giao diện dành cho giáo viên được xây dựng theo đúng tiến trình nghiệp vụ của sản phẩm. Ở lớp ngoài cùng là trang danh sách bài kiểm tra, nơi hiển thị các bài đã tạo dưới dạng thẻ, kèm theo tiêu đề, mô tả ngắn, trạng thái xử lý, trạng thái công bố và số lượt làm bài. Từ đây, giáo viên có thể khởi tạo bài kiểm tra mới hoặc tiếp tục làm việc với một bài đang có.

Trong môi trường biên tập, giáo viên có thể nhập nội dung bằng trí tuệ nhân tạo, rà soát lại đoạn văn nguồn, chỉnh sửa từng câu hỏi, thay đổi phương án trả lời, đáp án đúng và phần giải thích. Bên cạnh đó, hệ thống còn cung cấp trang thiết lập công bố để giáo viên sao chép liên kết, cho phép hay không cho phép làm bài ẩn danh, thiết lập chính sách làm lại, chọn mức phản hồi kết quả và cấu hình bộ đếm thời gian. Phần theo dõi kết quả được tách thành các màn hình tổng quan và danh sách lượt làm, hiển thị số liệu tổng hợp, phân bố điểm, nhóm người đạt

thành tích cao và chi tiết từng lượt nộp bài.



Hình 6.6: Giao diện biên tập bài kiểm tra dành cho giáo viên: hiển thị câu hỏi đã sinh tự động với các công cụ chỉnh sửa nội dung, phương án và cấp độ nhận thức

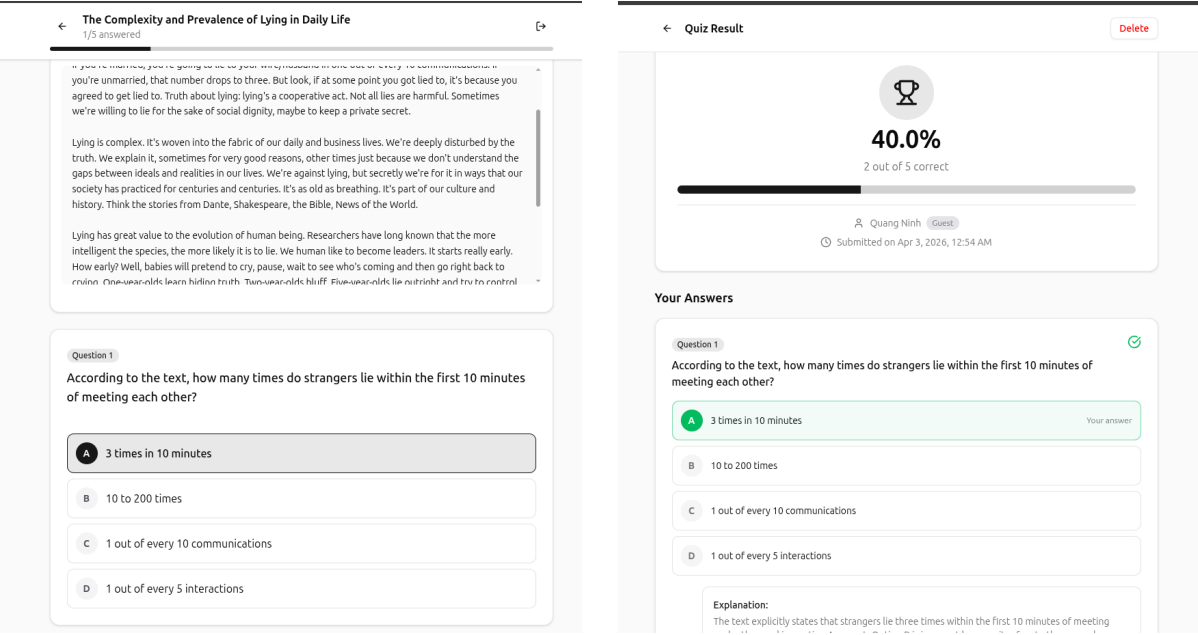
Hình 6.6 minh họa màn hình biên tập trung tâm của giáo viên. Thiết kế giao diện ưu tiên việc giữ cho vòng lặp chỉnh sửa ngắn và trực tiếp: giáo viên nhìn thấy ngay kết quả sinh tự động, có thể sửa từng câu hỏi tại chỗ và không phải chuyển qua nhiều màn hình trung gian. Cách tổ chức này phù hợp với mục tiêu của ứng dụng là hỗ trợ giáo viên hoàn thiện nhanh một bài kiểm tra trước khi công bố.

6.6.2 Giao diện dành cho người làm bài

Giao diện dành cho người làm bài được tối giản hơn đáng kể so với giao diện của giáo viên. Người dùng truy cập trực tiếp từ liên kết công khai, nhập tên hiển thị hoặc đăng nhập bằng tài khoản nếu cấu hình yêu cầu, sau đó đi vào màn hình làm bài. Trong quá trình làm bài, giao diện hiển thị rõ nội dung câu hỏi, các phương án lựa chọn, tiến độ làm bài và bộ đếm thời gian nếu giáo viên đã bật tính năng này.

Sau khi nộp bài, người dùng được chuyển tới trang kết quả. Nội dung của trang này không cố định mà thay đổi theo chính sách hiển thị đã cấu hình trước. Ở mức thấp nhất, hệ thống chỉ xác nhận đã ghi nhận bài làm. Ở các mức cao hơn, người làm bài có thể thấy điểm số, trạng thái đúng sai của từng câu, đáp án đúng và giải

thích chi tiết. Cơ chế này giúp cùng một ứng dụng có thể phục vụ cả tình huống kiểm tra nghiêm ngặt lẫn tình huống tự học có phản hồi đầy đủ.



Hình 6.7: Giao diện làm bài (trái) và trang kết quả (phải): người làm bài chọn đáp án với bộ đếm thời gian, sau đó nhận phản hồi theo mức hiển thị cấu hình bởi giáo viên

Hình 6.7 cho thấy cách Openlet tổ chức trải nghiệm của người làm bài theo hướng liền mạch. Màn hình làm bài ưu tiên khả năng tập trung và thao tác nhanh, trong khi màn hình kết quả ưu tiên tính rõ ràng của thông tin phản hồi. Sự phân tách này giúp người dùng không bị quá tải thông tin trong lúc làm bài, đồng thời vẫn nhận được mức hỗ trợ phù hợp sau khi hoàn thành.

Nhìn chung, lớp giao diện của Openlet phản ánh đúng triết lý thiết kế của toàn hệ thống: giáo viên được cung cấp một không gian biên soạn và kiểm soát linh hoạt, còn người làm bài tiếp cận một trải nghiệm gọn, rõ và an toàn. Đây là điều kiện cần để các kết quả nghiên cứu ở Chương 3 có thể được đưa vào sử dụng trong một ngữ cảnh nghiệp vụ thực tế.

Chương 7

Kết luận và Hướng phát triển

7.1 Tổng kết

Nghiên cứu này đã đề xuất và phát triển thành công một hệ thống tự động sinh câu hỏi trắc nghiệm đa cấp độ nhận thức từ dữ liệu ảnh tài liệu, giải quyết hai bài toán cốt lõi trong quy trình số hóa giáo dục: (1) OCR chính xác từ ảnh chụp chất lượng kém và (2) sinh MCQ theo thang nhận thức Bloom với độ khó và chất lượng sư phạm được kiểm soát chặt chẽ. Đóng góp nổi bật nhất của nghiên cứu là việc thiết kế một kiến trúc MAS sáng tạo, kết hợp sLLM với các cơ chế tự kiểm duyệt (tác tử Học sinh, tác tử Phân loại, và tác tử Hiệu chỉnh). Kết quả thực nghiệm cho thấy giải pháp này không chỉ khắc phục được hạn chế của sLLM khi hoạt động độc lập mà còn đạt được hiệu năng ngang ngửa với các LLM thương mại lớn ở cấp độ câu hỏi Nhớ và Hiểu, mở ra tiềm năng ứng dụng mạnh mẽ, tối ưu chi phí cho các môi trường tài nguyên hạn chế.

7.2 Hạn chế

Mặc dù đạt được những kết quả khả quan, nghiên cứu vẫn tồn tại một số hạn chế nhất định. Thứ nhất, hệ thống vẫn gặp khó khăn trong việc sinh các câu hỏi ở cấp độ Phân tích (Cấp 3), ngay cả khi có sự hỗ trợ của MAS, do giới hạn về năng lực suy luận nội tại của các sLLM hiện tại. Thứ hai, thực nghiệm hiện tại mới chỉ được tiến hành trên một tập dữ liệu tiếng Anh duy nhất (RACE), chưa được kiểm chứng đầy đủ trên các miền tri thức khác (như toán học, khoa học tự nhiên) hoặc các ngôn ngữ khác, đặc biệt là tiếng Việt. Thứ ba, phương pháp đánh giá hoàn toàn dựa trên cơ chế đánh giá tự động bằng mô hình (LLM-as-a-Judge), thiếu sự đối chiếu độc lập với đánh giá từ các chuyên gia sư phạm thực tế, điều này có thể tiềm ẩn các sai lệch chưa được nhận diện đầy đủ.

7.3 Hướng phát triển

Dựa trên các kết quả và hạn chế đã phân tích, nghiên cứu định hướng một số hướng phát triển cốt lõi trong tương lai:

- **Mở rộng đa ngôn ngữ và tiếng Việt:** Tích hợp và đánh giá hệ thống trên dữ liệu tiếng Việt, xây dựng các bộ lọc và tác tử chuyên biệt để xử lý đặc thù ngữ pháp và ngữ nghĩa, hướng tới triển khai thực tiễn trong nước.
- **Đánh giá bởi chuyên gia:** Tổ chức các đợt đánh giá độc lập với sự tham gia của giáo viên và chuyên gia giáo dục để thu thập phản hồi thực tế, từ đó tinh chỉnh bộ tiêu chí đánh giá tự động và cải thiện tính sư phạm của các phương án nhiều.
- **Tinh chỉnh mô hình:** Sử dụng tập dữ liệu MCQ chất lượng cao do MAS sinh ra để tinh chỉnh trực tiếp các sLLM. Hướng đi này kỳ vọng sẽ giúp sLLM nội tại hóa khả năng sinh câu hỏi khó, giảm sự phụ thuộc vào kiến trúc MAS tồn kém trong giai đoạn triển khai.
- **Tích hợp hệ thống học tập thích ứng:** Đưa mô-đun sinh câu hỏi vào một hệ thống quản lý học tập, cho phép tự động điều chỉnh độ khó của câu hỏi tiếp theo dựa trên năng lực và phản hồi tức thời của người học.

Tài liệu tham khảo

- [1] Marah Abdin, Jyoti Aneja, Harkirat Behl, Sébastien Bubeck, Ronen Eldan, Suriya Gunasekar, Michael Harrison, Russell J Hewett, Mojan Javaheripi, Piero Kauffmann, et al. Phi-4 technical report. *arXiv preprint arXiv:2412.08905*, 2024.
- [2] Lorin W Anderson and David R Krathwohl. *A taxonomy for learning, teaching, and assessing: A revision of Bloom’s taxonomy of educational objectives: complete edition*. Addison Wesley Longman, Inc., 2001.
- [3] Anthropic. Introducing claude haiku 4.5. <https://www.anthropic.com/news/claude-haiku-4-5>, 2025.
- [4] Benjamin S Bloom, Max D Engelhart, Edward J Furst, Walker H Hill, David R Krathwohl, et al. *Taxonomy of educational objectives: The classification of educational goals. Handbook 1: Cognitive domain*. Longman New York, 1956.
- [5] Harrison Chase. Langchain. <https://github.com/langchain-ai/langchain>, 2022.
- [6] Cheng Cui, Ting Sun, Suyin Liang, Tingquan Gao, Zelun Zhang, Jiaxuan Liu, Xueqing Wang, Changda Zhou, Hongen Liu, Manhui Lin, Yue Zhang, Yubo Zhang, Handong Zheng, Jing Zhang, Jun Zhang, Yi Liu, Dianhai Yu, and Yanjun Ma. Paddleocr-vl: Boosting multilingual document parsing via a 0.9b ultra-compact vision-language model, 2025.
- [7] Sabina Elkins, Ekaterina Kochmar, Iulian Serban, and Jackie CK Cheung. How useful are educational questions generated by large language models? In *International Conference on Artificial Intelligence in Education*, pages 536–542. Springer, 2023.
- [8] Cosimo Galeone, Minsu Park, Giuseppe Ettorre, and Daniele Ligorio. When correct isn’t usable: Improving structured output reliability in small language models. *arXiv preprint arXiv:2605.02363*, 2026.
- [9] Google. Google fonts. <https://fonts.google.com/>, 2010.
- [10] Google. Firebase. <https://firebase.google.com/>, 2011.

- [11] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024.
- [12] Alexander Groleau, Kok Wei Chee, Stefan Larson, Samay Maini, and Jonathan Boarman. Augraphy: A data augmentation library for document images. 2022.
- [13] Michael Heilman and Noah A Smith. Good question! statistical ranking for question generation. In *Human language technologies: The 2010 annual conference of the North American Chapter of the Association for Computational Linguistics*, pages 609–617, 2010.
- [14] Guokun Lai, Qizhe Xie, Hanxiao Liu, Yiming Yang, and Eduard Hovy. Race: Large-scale reading comprehension dataset from examinations. In *Proceedings of the 2017 conference on empirical methods in natural language processing*, pages 785–794, 2017.
- [15] LangChain Team. LangGraph: Build stateful, multi-actor applications with LLMs. <https://langchain-ai.github.io/langgraph/>, 2024. Documentation and framework reference.
- [16] Yumeng Li, Guang Yang, Hao Liu, Bowen Wang, and Colin Zhang. dots.ocr: Multilingual document layout parsing in a single vision-language model, 2025.
- [17] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. G-eval: Nlg evaluation using gpt-4 with better human alignment. In *Proceedings of the 2023 conference on empirical methods in natural language processing*, pages 2511–2522, 2023.
- [18] Meta Platforms, Inc. React - a javascript library for building user interfaces. <https://react.dev/>, 2013.
- [19] Ruslan Mitkov, Ha Le An, and Nikiforos Karamanis. A computer-aided environment for generating multiple-choice test items. *Natural language engineering*, 12(2):177–194, 2006.
- [20] Hiroyuki Miura. Pyppeteer: Headless chrome/chromium automation library (unofficial port of puppeteer). <https://github.com/pyppeteer/pyppeteer>, 2018.

- [21] Steven Moore, Huy A Nguyen, Tianying Chen, and John Stamper. Assessing the quality of multiple-choice questions using gpt-4 and rule-based methods. In *European conference on technology enhanced learning*, pages 229–245. Springer, 2023.
- [22] Junbo Niu, Zheng Liu, Zhuangcheng Gu, Bin Wang, Linke Ouyang, Zhiyuan Zhao, Tao Chu, Tianyao He, Fan Wu, Qintong Zhang, Zhenjiang Jin, et al. Mineru2.5: A decoupled vision-language model for efficient high-resolution document parsing, 2025.
- [23] Novita AI. Novita ai. <https://novita.ai/>, 2023.
- [24] NVIDIA. Nvidia nemotron nano 2: An accurate and efficient hybrid mamba-transformer reasoning model, 2025.
- [25] OpenRouter. Openrouter. <https://openrouter.ai/>, 2023.
- [26] Pranav Rajpurkar, Jian Zhang, Konstantin Lopyrev, and Percy Liang. Squad: 100,000+ questions for machine comprehension of text. In *Proceedings of the 2016 conference on empirical methods in natural language processing*, pages 2383–2392, 2016.
- [27] Aaditya Singh, Adam Fry, Adam Perelman, Adam Tart, Adi Ganesh, Ahmed El-Kishky, Aidan McLaughlin, Aiden Low, AJ Ostrow, Akhila Ananthram, et al. Openai gpt-5 system card. *arXiv preprint arXiv:2601.03267*, 2025.
- [28] Ray Smith. An overview of the tesseract ocr engine. In *Ninth international conference on document analysis and recognition (ICDAR 2007)*, volume 2, pages 629–633. IEEE, 2007.
- [29] Gemini Team, Rohan Anil, Sebastian Borgeaud, Jean-Baptiste Alayrac, Jiahui Yu, Radu Soricut, Johan Schalkwyk, Andrew M Dai, Anja Hauth, Katie Millican, et al. Gemini: a family of highly capable multimodal models. *arXiv preprint arXiv:2312.11805*, 2023.
- [30] Gemma Team, Aishwarya Kamath, Johan Ferret, Shreya Pathak, Nino Vieillard, Ramona Merhej, Sarah Perrin, Tatiana Matejovicova, Alexandre Ramé, Morgane Rivi re, Louis Rouillard, Thomas Mesnard, Geoffrey Cideron, Jean bastien Grill, Sabela Ramos, Edouard Yvinec, Michelle Casbon, Etienne Pot,

Ivo Penchev, Gaël Liu, Francesco Visin, Kathleen Kenealy, Lucas Beyer, Xiaohai Zhai, Anton Tsitsulin, Robert Busa-Fekete, Alex Feng, Noveen Sachdeva, Benjamin Coleman, Yi Gao, Basil Mustafa, Iain Barr, Emilio Parisotto, David Tian, Matan Eyal, Colin Cherry, Jan-Thorsten Peter, Danila Sinopalnikov, Surya Bhupatiraju, Rishabh Agarwal, Mehran Kazemi, Dan Malkin, Ravin Kumar, David Vilar, Idan Brusilovsky, Jiaming Luo, Andreas Steiner, Abe Friesen, Abhanshu Sharma, Abheesht Sharma, Adi Mayrav Gilady, Adrian Goedeckemeyer, Alaa Saade, Alex Feng, Alexander Kolesnikov, Alexei Bendebury, Alvin Abdagic, Amit Vadi, András György, André Susano Pinto, Anil Das, Ankur Bapna, Antoine Miech, Antoine Yang, Antonia Paterson, Ashish Shenoy, Ayan Chakrabarti, Bilal Piot, Bo Wu, Bobak Shahriari, Bryce Petrini, Charlie Chen, Charline Le Lan, Christopher A. Choquette-Choo, CJ Carey, Cormac Brick, Daniel Deutsch, Danielle Eisenbud, Dee Cattle, Derek Cheng, Dimitris Paparas, Divyashree Shivakumar Sreepathihalli, Doug Reid, Dustin Tran, Dustin Zelle, Eric Noland, Erwin Huizenga, Eugene Kharitonov, Frederick Liu, Gagik Amirkhanyan, Glenn Cameron, Hadi Hashemi, Hanna Klimczak-Plucińska, Harman Singh, Harsh Mehta, Harshal Tushar Lehri, Hussein Hazimeh, Ian Ballantyne, Idan Szpektor, Ivan Nardini, Jean Pouget-Abadie, Jetha Chan, Joe Stanton, John Wieting, Jonathan Lai, Jordi Orbay, Joseph Fernandez, Josh Newlan, Ju yeong Ji, Jyotinder Singh, Kat Black, Kathy Yu, Kevin Hui, Kiran Vodrahalli, Klaus Greff, Linhai Qiu, Marcella Valentine, Marina Coelho, Marvin Ritter, Matt Hoffman, Matthew Watson, Mayank Chaturvedi, Michael Moynihan, Min Ma, Nabila Babar, Natasha Noy, Nathan Byrd, Nick Roy, Nikola Momchev, Nilay Chauhan, Noveen Sachdeva, Oskar Bunyan, Pankil Botarda, Paul Caron, Paul Kishan Rubenstein, Phil Culliton, Philipp Schmid, Pier Giuseppe Sessa, Pingmei Xu, Piotr Stanczyk, Pouya Tafti, Rakesh Shivanna, Renjie Wu, Renke Pan, Reza Rokni, Rob Willoughby, Rohith Vallu, Ryan Mullins, Sammy Jerome, Sara Smoot, Sertan Girgin, Shariq Iqbal, Shashir Reddy, Shruti Sheth, Siim Põder, Sijal Bhatnagar, Sindhu Raghuram Panyam, Sivan Eiger, Susan Zhang, Tianqi Liu, Trevor Yacovone, Tyler Liechty, Uday Kalra, Utku Evci, Vedant Misra, Vincent Roseberry, Vlad Feinberg, Vlad Kolesnikov, Woohyun Han, Woosuk Kwon, Xi Chen, Yinlam Chow, Yuvein Zhu, Zichuan Wei, Zoltan Egyed, Victor Cotruta, Minh Giang, Phoebe Kirk, Anand Rao, Kat

Black, Nabila Babar, Jessica Lo, Erica Moreira, Luiz Gustavo Martins, Omar Sanseviero, Lucas Gonzalez, Zach Gleicher, Tris Warkentin, Vahab Mirrokni, Evan Senter, Eli Collins, Joelle Barral, Zoubin Ghahramani, Raia Hadsell, Yossi Matias, D. Sculley, Slav Petrov, Noah Fiedel, Noam Shazeer, Oriol Vinyals, Jeff Dean, Demis Hassabis, Koray Kavukcuoglu, Clement Farabet, Elena Buchatskaya, Jean-Baptiste Alayrac, Rohan Anil, Dmitry, Lepikhin, Sebastian Borgeaud, Olivier Bachem, Armand Joulin, Alek Andreev, Cassidy Hardin, Robert Dadashi, and Léonard Hussenot. Gemma 3 technical report, 2025.

- [31] Vercel. Next.js by vercel - the react framework. <https://nextjs.org/>, 2016.
- [32] Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, Dawei Zhu, Binghuai Lin, Yunbo Cao, Lingpeng Kong, Qi Liu, Tianyu Liu, et al. Large language models are not fair evaluators. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 9440–9450, 2024.
- [33] Haoran Wei, Yaofeng Sun, and Yukun Li. Deepseek-ocr 2: Visual causal flow. *arXiv preprint arXiv:2601.20552*, 2026.
- [34] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.
- [35] Weijie Xu, Shixian Cui, Xi Fang, Chi Xue, Stephanie Eckman, and Chandan K Reddy. Sata-bench: Select all that apply benchmark for multiple choice questions. *arXiv preprint arXiv:2506.00643*, 2025.
- [36] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [37] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. React: Synergizing reasoning and acting in language models. *arXiv preprint arXiv:2210.03629*, 2022.